Gaussian Filtering and Edges

Pablo Mesejo

pmesejo@go.ugr.es

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD DE GRANADA





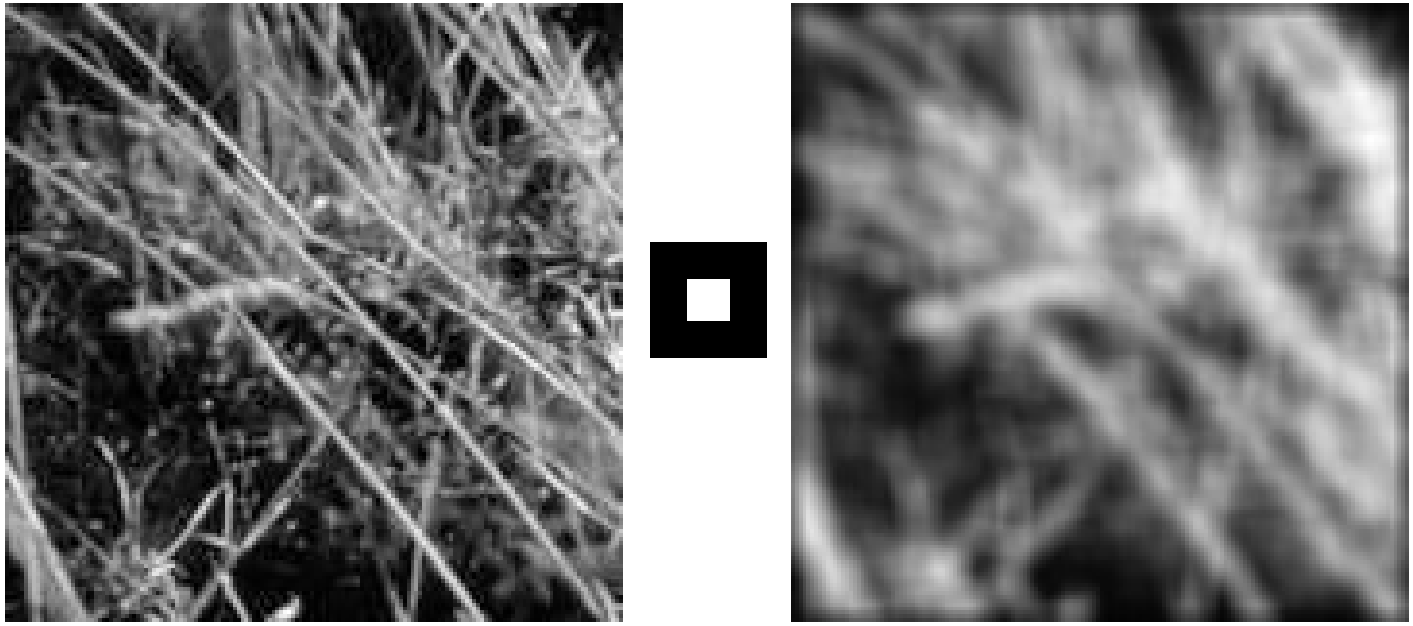
DaSCI

Instituto Andaluz de Investigación en **Data Science** and **Computational Intelligence**

Index

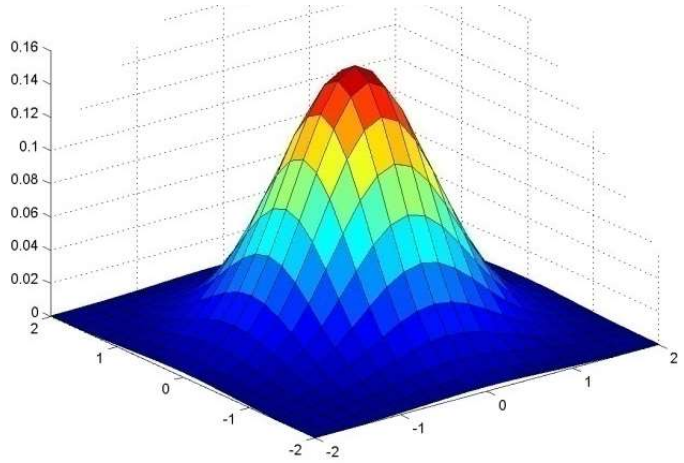
- Smoothing with a box filter vs with a Gaussian filter
- Gaussian kernel discretization
- Gaussian properties and filter separability
- Computing derivatives: high-frequency features (high-pass filters)
  - First-order derivatives
  - Second-order derivatives (the Laplacian of Gaussian)
- The Canny Edge Detector

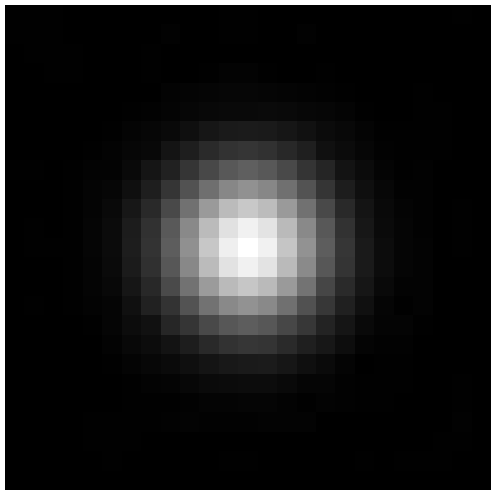
Smoothing with box filter



Source: D. Forsyth

Gaussian kernel



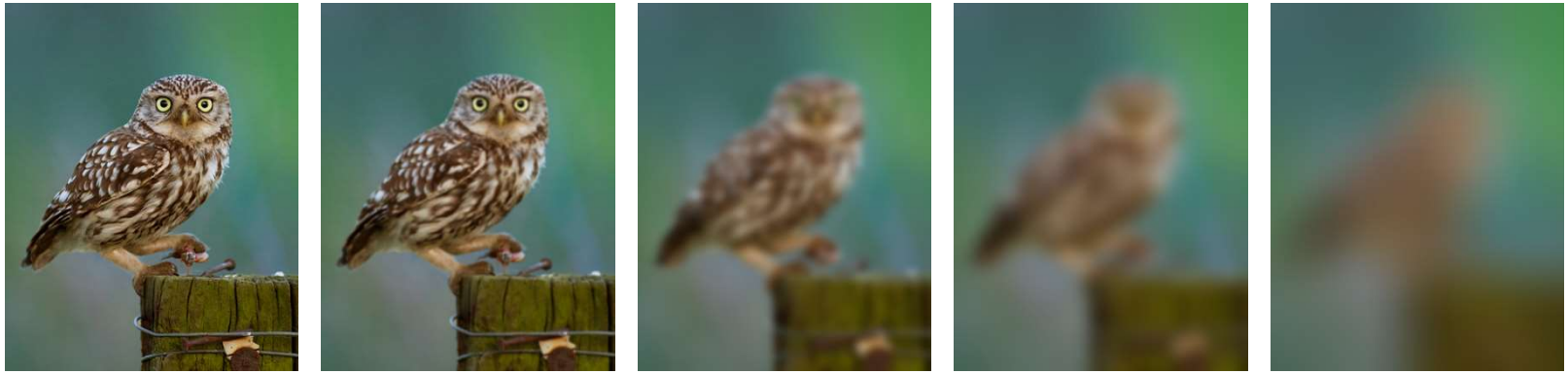


An isotropic (circularly symmetric) filter

$$G_\sigma = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

- The coefficients are a 2D Gaussian.
- Gives more weight to the central pixels and less weight to the neighbors.
- The farther away the neighbors, the smaller the weight.

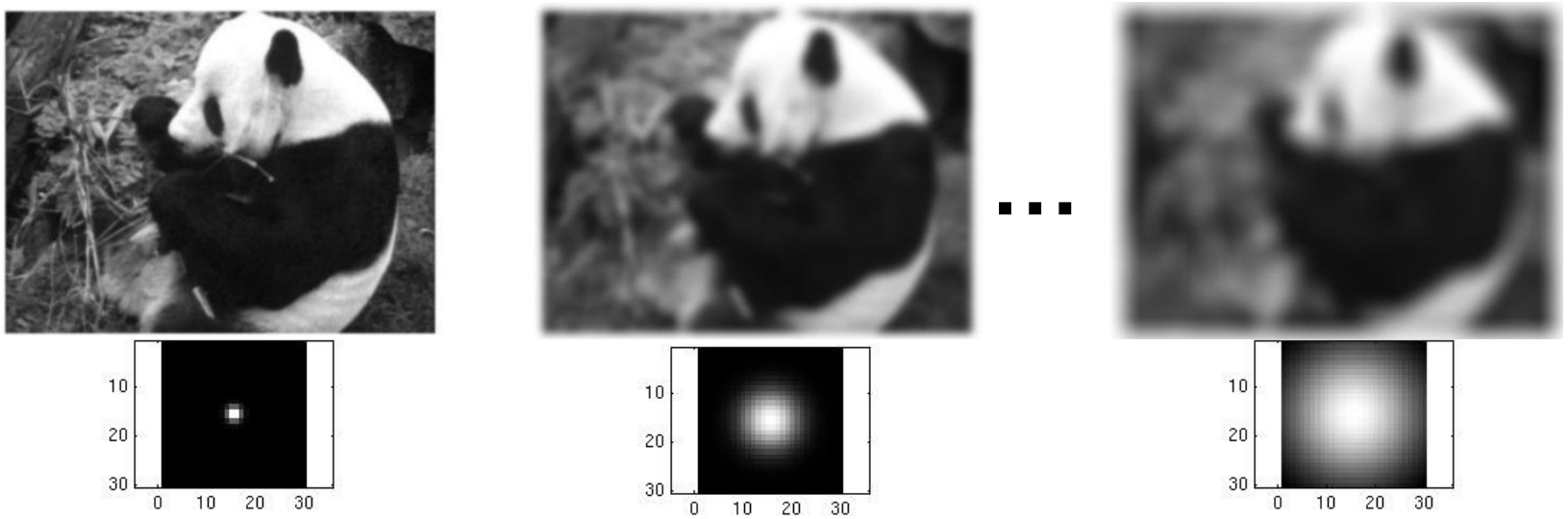
Gaussian filters



$\sigma = 1$ $\sigma = 5$ $\sigma = 10$ $\sigma = 30$

Smoothing with a Gaussian

Parameter σ is the “scale” / “width” / “spread” of the Gaussian kernel and controls the amount of smoothing.




```
sigma=0
for ksize in np.arange(3, 9, 2):
        kernel=cv2.getGaussianKernel(ksize, sigma)
        outim=cv2.filter2D(im,-1,kernel@kernel.T)
        cv2.imshow(outim)
```

If it’s non-positive, it’s computed from ksize

If you use Google Colab, `from google.colab.patches import cv2_imshow`

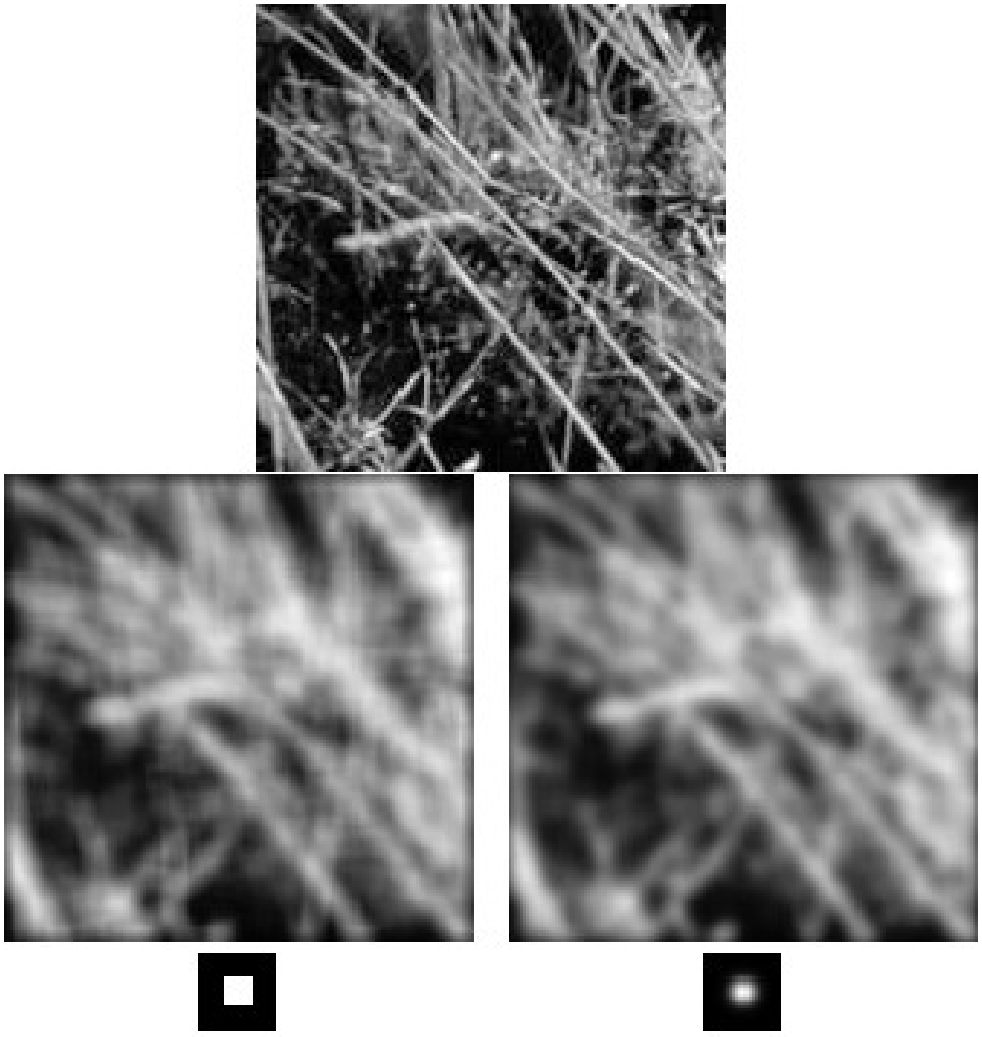
When ddepth=-1, the output image will have the same depth/precision as the source

We create the 2D kernel from the 1D kernel

Depth combinations

Input depth ( )	Output depth (ddepth)
CV_8U	-1/CV_16S/CV_32F/CV_64F
CV_16U/CV_16S	-1/CV_32F/CV_64F
CV_32F	-1/CV_32F
CV_64F	-1/CV_64F

Mean vs. Gaussian filtering



The **box filter** provides **edgy artifacts!**

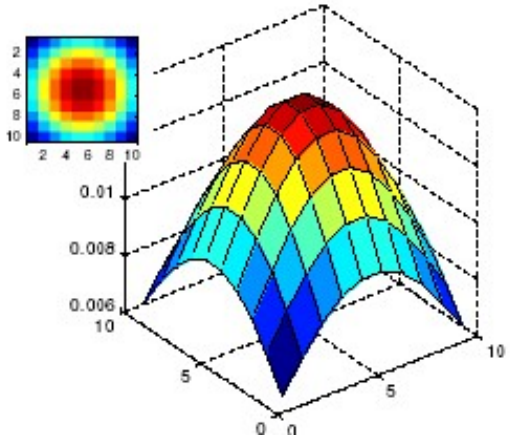
Gaussian is a better low-pass filter → it does not create high-frequency artifacts.

Gaussian filter

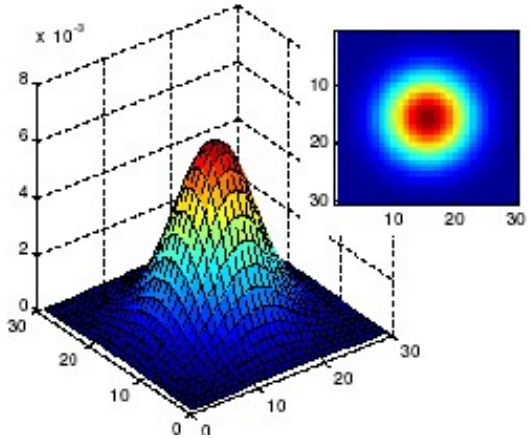
Which of the three discretizations/approximations below do you think is better?



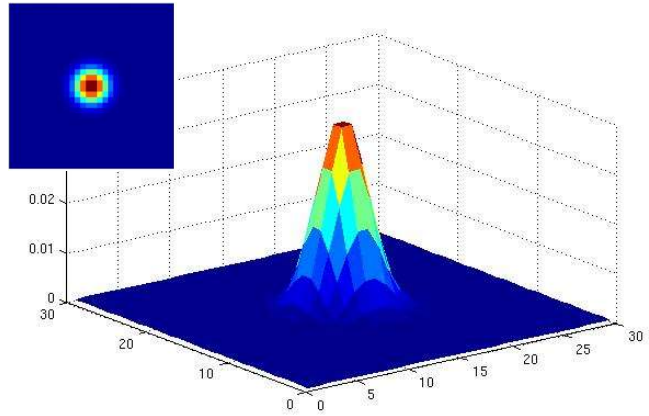
- What parameters matter here?
  - **Size** of kernel or mask
  - **Standard deviation (sigma)** of Gaussian determines extent of smoothing



σ = 5 with 11 x 11 kernel



σ = 5 with 31 x 31 kernel

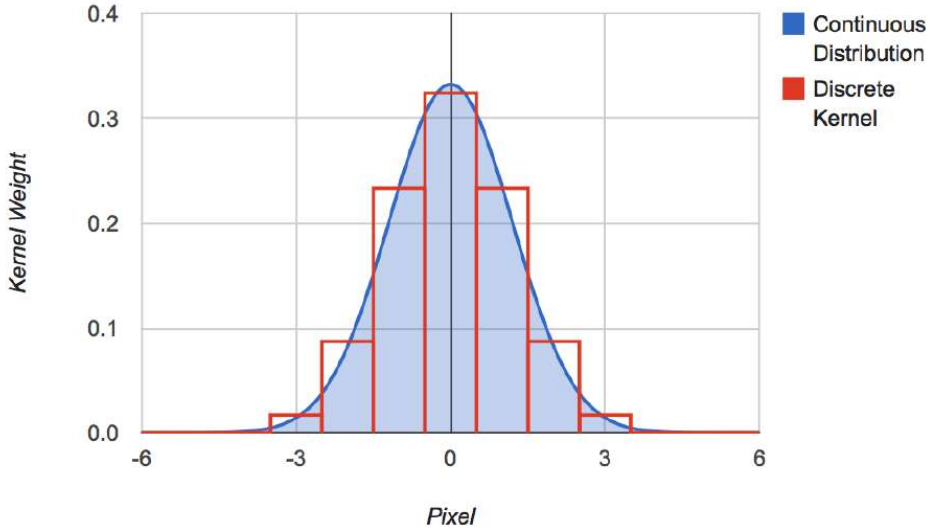


σ = 2 with 31 x 31 kernel

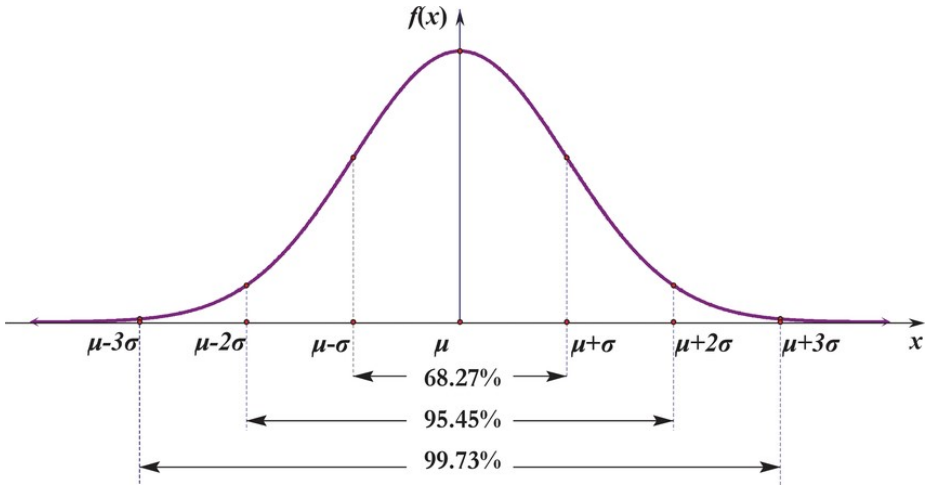
Gaussian discretization

We want to discretize a Gaussian.

There are different approaches to this discretization. Here, we will consider one of them.



We know that **almost all values are within 3 standard deviations of the mean** → It makes sense to use, **by default, 3σ.**



Gaussian discretization

$$f(x,\sigma) = c \cdot e^{-\frac{x^2}{2\sigma^2}}$$

We ignore this normalizing constant ($^1/_{\sigma\sqrt{2\pi}}$)**!** The shape of the filter is the same! We later normalize the kernel, making the sum of the coefficients equal to 1

Gaussian discrete mask 1D:
$[f(-k), \dots, f(-1), f(0), f(1), \dots, f(k)]$, k *an integer*

If σ= 1 → kernel size is 7
(ie, k = 3)

According to the Gaussian properties, $\min(k) \geq 3\sigma$ provides samples covering more than 99% of the area under the curve

We enforce $\sum_{i=-k}^{k} f(i) = 1$

The mask must add up to 1 → you must divide your mask by the sum of its coefficients

Why?? 🤔

Gaussian discretization

We can go from σ to mask size (T), or viceversa, in order to get the Gaussian mask

If T is the mask size, $T = 2 \cdot k + 1 \rightarrow k = (T-1)/2 \rightarrow (T-1)/2 = 3 \cdot \sigma \rightarrow 2 \cdot [3 \cdot \sigma] + 1 = T$

Let's see an example. We have T=5:

- We'd have 5 positions: [f(-2), f(-1), f(0), f(1), f(2)]
- We compute the suitable sigma: $(T-1)/2 = 3 \cdot \sigma \rightarrow (5-1)/2 = 3 \cdot \sigma \rightarrow 2 = 3 \cdot \sigma \rightarrow \sigma = 2/3 = 0.67$
- We compute the values of the mask substituting $x = \{-2, -1, 0, 1, 2\}$ on $f(x) = e^{-\frac{x^2}{2 \cdot 0.67^2}}$

Once we have σ and *k* we can discretize the mask by applying the Gaussian function (or its derivatives)

Note: the discretization of a continuous function is just an approximation. There is no single way of doing it! See, for instance, Stan Birchfield's chapter 5.2.4 on constructing Gaussian kernels, or the way in which OpenCV operates when using `getGaussianKernel()`.

Gaussian discretization: finite approximation

Convolving a box filter with itself yields an approximation to a Gaussian kernel (easy way to construct a Gaussian kernel):

$$\frac{1}{2}\begin{bmatrix}1 & 1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1 & 1\end{bmatrix} = \frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix}$$

$$\frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix} * \frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix} = \frac{1}{16}\begin{bmatrix}1 & 4 & 6 & 4 & 1\end{bmatrix}$$

These are the odd rows of the binomial (Pascal's) triangle:

k=1, σ^2=0.5, σ=0.7

k=2, σ^2=1, σ=1

$$\begin{array}{ccccc} & & 1 & & \\ & 1 & & 1 & \\ \boxed{1} & \boxed{2} & & \boxed{1} & \\ 1 & 3 & 3 & 1 & \\ \boxed{1} & \boxed{4} & \boxed{6} & \boxed{4} & \boxed{1} \end{array}$$

$(2k+1)^{th}$ row approximates Gaussian with $\sigma^2=k/2$

Check Stan Birchfield's book (Chapter 5.2) for more information regarding the binomial and trinomial triangle to generate Gaussian approximations

Gaussian Properties

- Remove “high-frequency” components from the image (*low-pass filter*). All values are positive and sum up to 1.
  - Images become smoother.

- Completely described by 1st (mean) and 2nd (variance) order statistics (moments).

- The Fourier Transform of a Gaussian function is also a Gaussian

$$f(x) = e^{-\frac{x^2}{2\sigma^2}} \quad \Rightarrow \quad \mathcal{F}[f(x)] = \sqrt{2\pi}\sigma e^{-2\pi^2\sigma^2\omega^2}$$

Gaussian Properties

- Closed under convolution (convolution of two Gaussians is another Gaussian)



  - When you convolve a Gaussian function (G_1) with another Gaussian function (G_2), the result is still a Gaussian function (G)
  - Convolving twice with Gaussian kernel of width σ is the same as convolving once with kernel of width $\sqrt{\sigma^2 + \sigma^2} = \sqrt{2\sigma^2} = \sigma\sqrt{2}$
  - In general, n repeated convolutions with a Gaussian of width σ are equivalent to a single convolution with a Gaussian of width $\sigma\sqrt{n}$

$$G(x, y|\sigma) = G\left(x, y \middle| \sqrt{\sigma_1^2 + \sigma_2^2}\right) = G_1(x, y|\sigma_1) * G_2(x, y|\sigma_2)$$

https://en.wikipedia.org/wiki/Sum_of_normally_distributed_random_variables#Proof_using_convolutions

Gaussian Properties

- Separable (efficient)
  - Factors into product of two 1D Gaussians
  - Example:

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$



```
import scipy.signal as sp
import numpy as np

np.outer(np.array([1,2,1]), np.array([1,2,1]))

sp.convolve2d(np.array([[0,1,0],[0,2,0], [0,1,0]]),
              np.array([[0,0,0],[1,2,1], [0,0,0]]), mode='same')
```

Outer product:

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_m \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

$$\mathbf{u} \otimes \mathbf{v} = \mathbf{A} = \begin{bmatrix} u_1 v_1 & u_1 v_2 & \dots & u_1 v_n \\ u_2 v_1 & u_2 v_2 & \dots & u_2 v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_m v_1 & u_m v_2 & \dots & u_m v_n \end{bmatrix}$$

Separability of the Gaussian

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2}\exp^{-\frac{x^2+y^2}{2\sigma^2}}$$

$$= \left(\frac{1}{\sqrt{2\pi}\sigma}\exp^{-\frac{x^2}{2\sigma^2}}\right)\left(\frac{1}{\sqrt{2\pi}\sigma}\exp^{-\frac{y^2}{2\sigma^2}}\right)$$

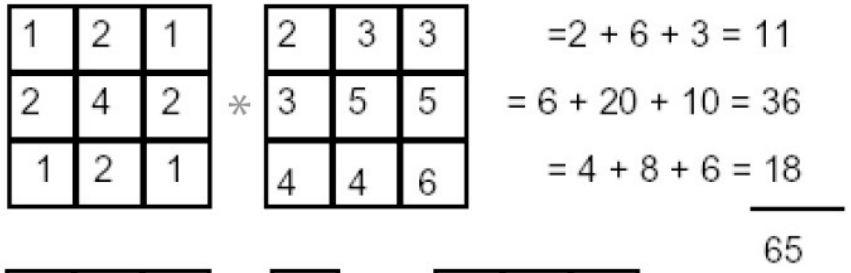
The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian

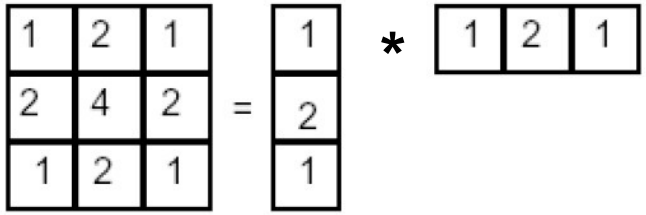
Source: D. Lowe

Separability example

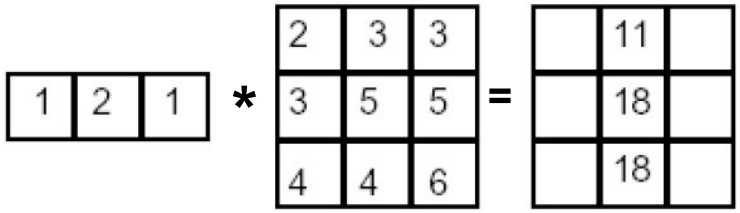
2D filtering (center location only)



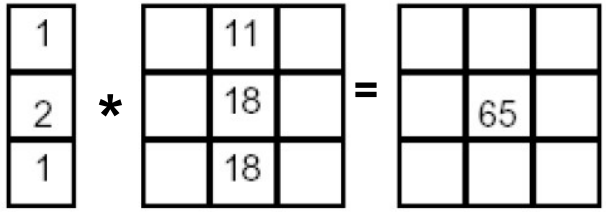
The filter factors into a product of 1D filters:



Perform filtering along rows:



Followed by filtering along the remaining column:



Source: K. Grauman

Separability example

We combine information from different rows
(kernel Y - Coefficients for filtering each column)

We combine information from different columns
(kernel X - Coefficients for filtering each row)

```
[1]
[2]
[1]
```

```
[ 0,  0,  0,  0,  0]
[ 0, 10, 20, 30,  0]
[ 0, 40, 50, 60,  0]
[ 0, 70, 80, 90,  0]
[ 0,  0,  0,  0,  0]
```

```
[1, 2, 1]
```

```
[  0,   0,   0,   0,   0]
[  0,  60,  90, 120,   0]
[  0, 160, 200, 240,   0]
[  0, 180, 210, 240,   0]
[  0,   0,   0,   0,   0]
```

Input Image

```
[ 0,  0,  0,  0,  0]
[ 0, 10, 20, 30,  0]
[ 0, 40, 50, 60,  0]
[ 0, 70, 80, 90,  0]
[ 0,  0,  0,  0,  0]
```

Kernel

```
[1, 2, 1]
```

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Separable Kernel

http://www.songho.ca/dsp/convolution/convolution2d_separable.html

```
[  0,   0,   0,   0,   0]
[  0, 210, 360, 330,   0]
[  0, 520, 800, 680,   0]
[  0, 570, 840, 690,   0]
[  0,   0,   0,   0,   0]
```

Why is separability useful in practice?

- Separability means that a 2D convolution can be reduced to two 1D convolutions (one along rows and one along columns)
- What is the complexity of filtering an $n \times n$ image with an $m \times m$ kernel?
  - $O(n^2 m^2)$

We move the $m \times m$ filter through the whole image. So, we do $m \cdot m$ multiplications for each of the $n \cdot n$ pixels in the image: $m^2 \cdot n^2$ multiplications

- What if the kernel is separable?
  - $O(n^2 m)$

We move the $1 \times m$ filter through the whole image. So, we do m multiplications for each of the $n \cdot n$ pixels in the image (one along rows and another along columns): $m \cdot n \cdot n + m \cdot n \cdot n = 2 \cdot (m \cdot n^2)$

Example: 512x512 image, 11x11 filter

- 1 convolution 2D:	512*512*11*11	~32M multiplications
- 2 convolutions 1D:	512*512*11 + 512*512*11	~6M multiplications

More ideas about separability

- Allowed because convolution is associative ($I * (v * h) = (I * v) * h$)
- Some 2D kernels can be directly decomposed into 1D kernels:

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix} = \frac{1}{4}\begin{bmatrix}1\\ 2\\ 1\end{bmatrix} * \frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix}$$

$$= \frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix} * \frac{1}{4}\begin{bmatrix}1\\ 2\\ 1\end{bmatrix}$$

$$\frac{1}{3}\begin{bmatrix}1\\ 1\\ 1\end{bmatrix} * \frac{1}{3}\begin{bmatrix}1 & 1 & 1\end{bmatrix} = \frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$$

- A 2D kernel is *separable* iff every pair of nonzero rows/cols is linearly dependent
  - This filter $\begin{bmatrix}0 & 1 & 0\\ 1 & -4 & 1\\ 0 & 1 & 0\end{bmatrix}$ is not separable:

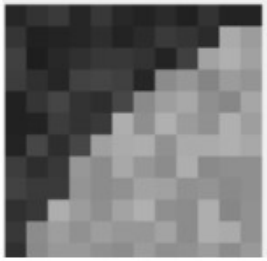
$\alpha \cdot [0, 1, 0] + \beta \cdot [1, -4, 1] = [0, 0, 0] \leftrightarrow \alpha = 0 \wedge \beta = 0 \rightarrow$ Linearly independent vectors!!

Note: any 2D-mask admits a SVD decomposition (*low-rank approximations*): sum of several separable kernels.
https://bartwronski.com/2020/02/03/separate-your-filters-svd-and-low-rank-approximation-of-image-filters/

Computing derivatives: high-frequency features (high-pass filters)

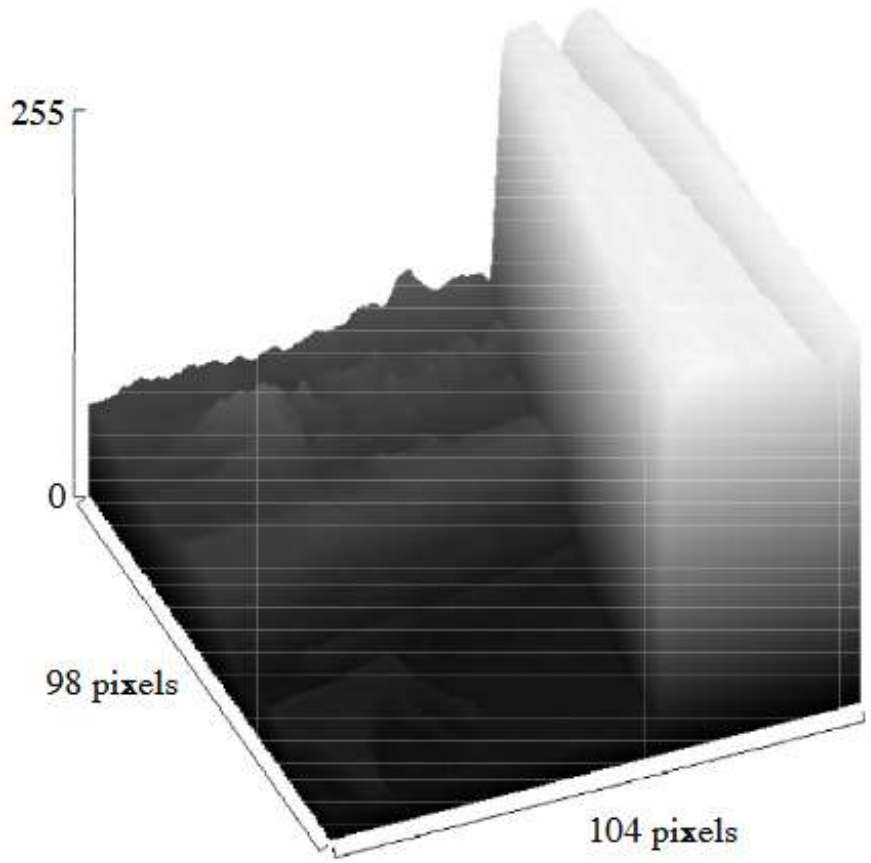
Edges

	24	25	26	27	28	29	30	31	32	33	34	35
24	15	20	23	15	22	16	14	17	13	20	14	14
25	21	13	14	15	18	14	17	23	21	17	67	61
26	25	12	16	19	22	22	20	14	25	64	69	63
27	24	19	15	26	27	25	15	55	59	52	61	58
28	13	21	27	20	18	28	54	62	66	56	53	65
29	13	14	24	20	23	68	56	69	61	57	69	62
30	14	28	18	28	63	67	68	57	67	67	69	66
31	24	18	22	59	55	66	59	55	68	54	56	56
32	26	23	23	58	55	57	64	63	57	58	54	56
33	19	22	68	61	56	64	69	56	55	68	53	57
34	22	55	62	57	62	59	59	64	55	68	67	57
35	19	54	60	60	57	55	61	56	57	61	55	60



https://cs.wellesley.edu/~cs332/doc/edgeNotes.pdf

Edges



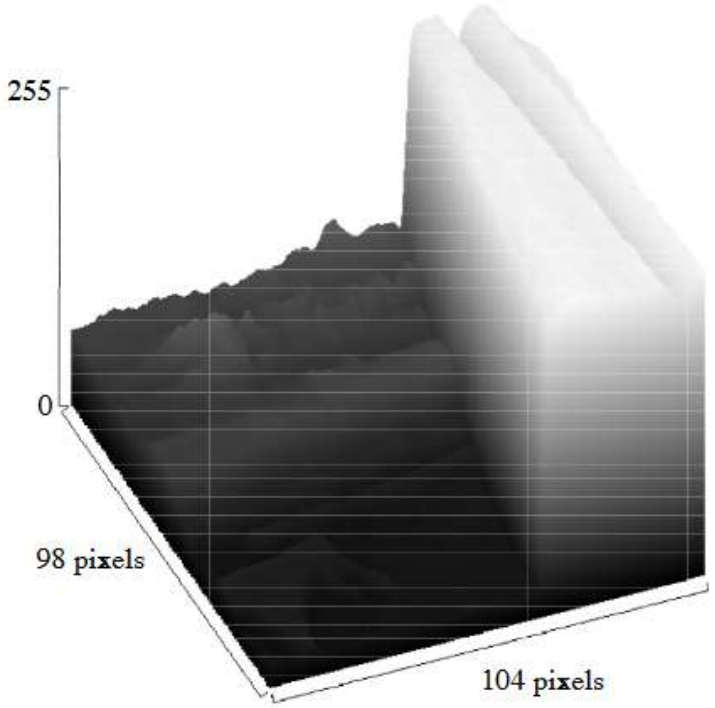
Edge: an abrupt change in pixel intensity.

Contour: a closed curve that joins edge points to represent the shape of an object.

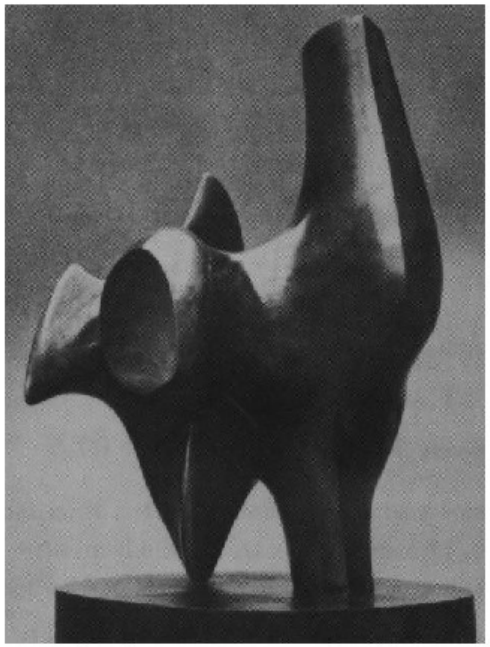
https://www.cs.auckland.ac.nz/~rklette/TeachAuckland.html/775/pdfs/D02-Images.pdf

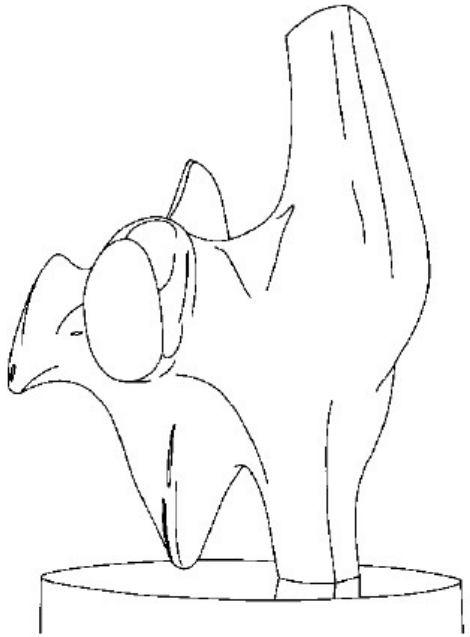
Edges and High Frequencies

- Frequency in 2D images:
  - rate of change of image intensity with respect to spatial coordinates
    - If it "involves" many pixels to make a change
      → low frequency
    - If it "involves" few pixels to make a change (i.e., the change is abrupt)
      → high frequency



Edge detection





- **Convert a 2D image into a set of curves**
  - **Extracts salient features of the scene**
  - **More compact than pixels**

Importance of intensity edges



from Walther et al., Simple line drawings suffice for functional MRI decoding of natural scene categories, PNAS 2011

→ Much information is retained by the edges

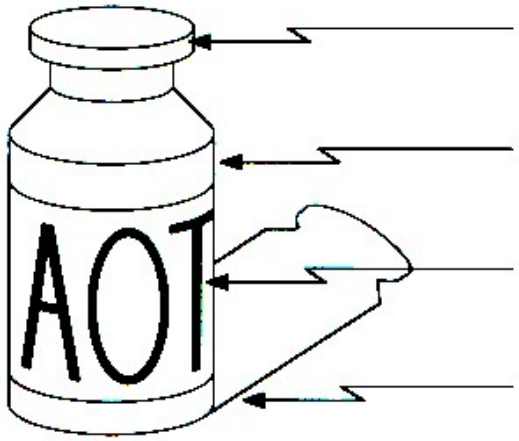
Importance of intensity edges



from Walther et al., Simple line drawings suffice for functional MRI decoding of natural scene categories, PNAS 2011

→ Much information is retained by the edges

Origin of edges



surface normal discontinuity (top vs side)

depth discontinuity (side of an object)

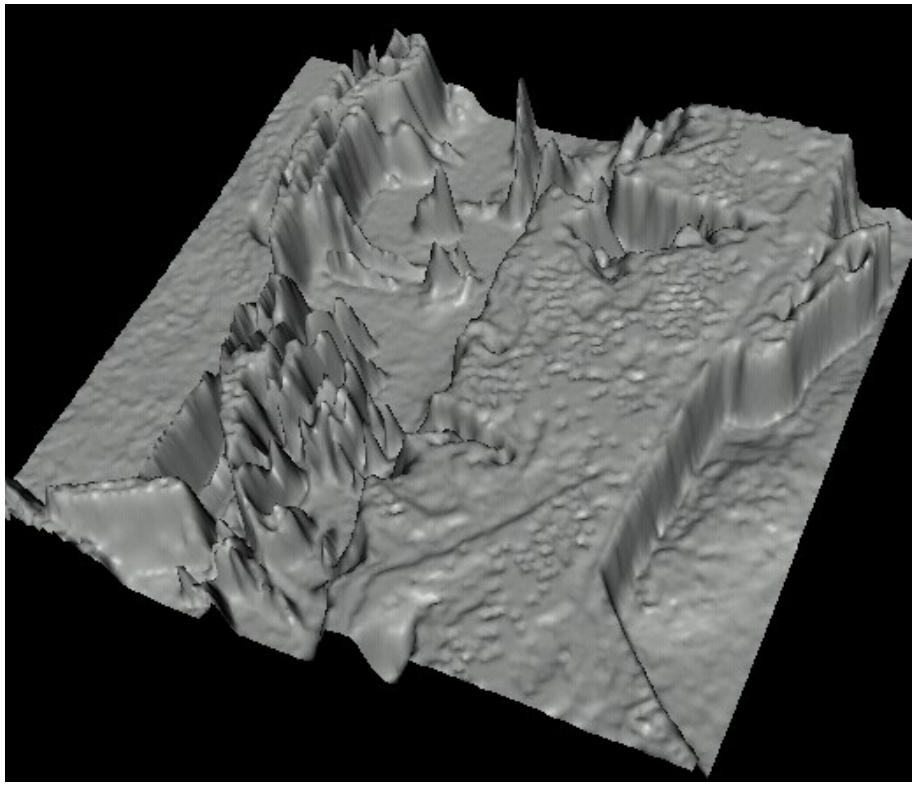
surface **color/reflectance** discontinuity (text/ink)

illumination discontinuity (shadows)

- Edges are caused by a variety of factors

Images as functions…

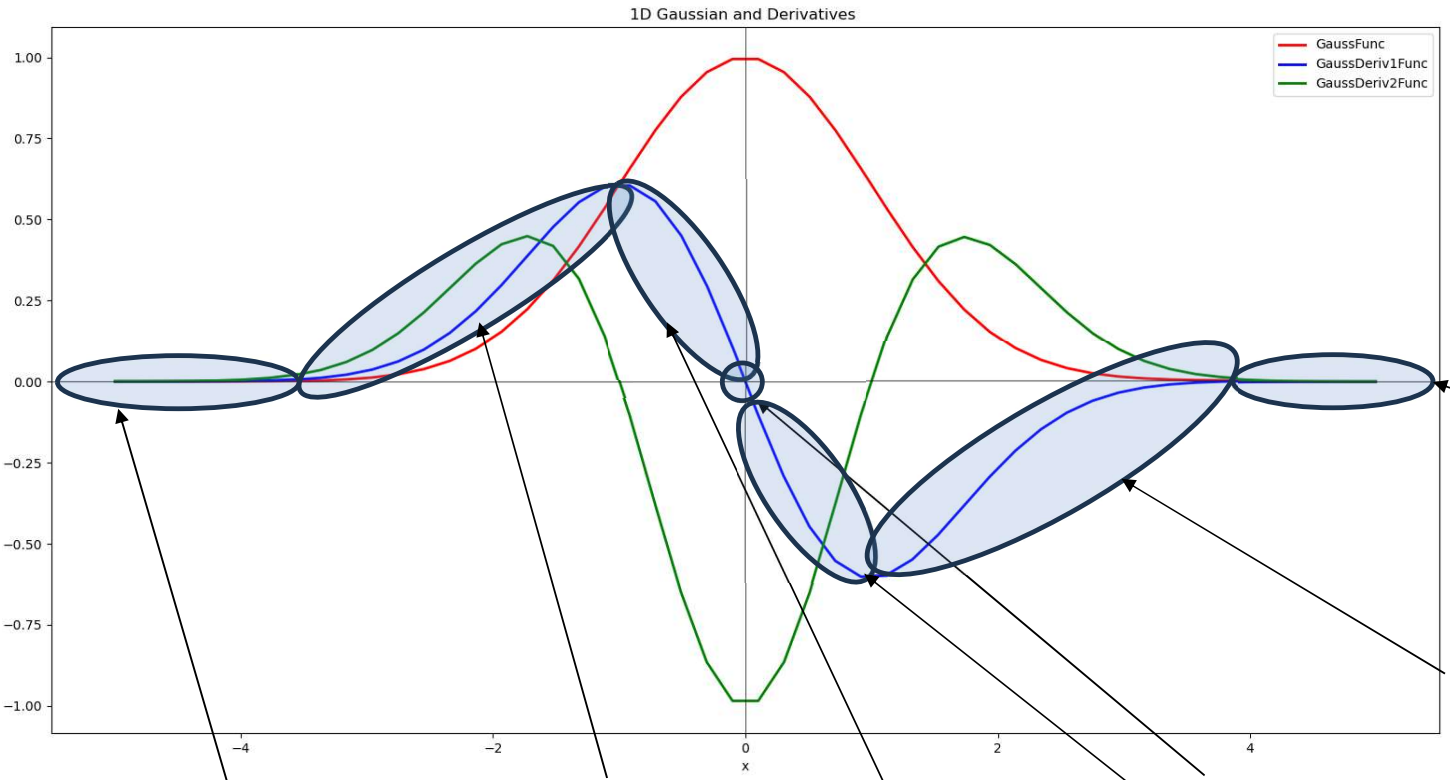




- Edges look like steep cliffs

Remembering the concept of derivative

The derivative shows the **instantaneous and local sensitivity** of a **function's output with respect to small changes in the input**.



Remember that **the derivative** of a function at a chosen input value, when it exists, **is the slope of the tangent line** of the function **at that point**.

No variation → derivative=0

Decreasing negative slope

Increasing positive slope

Slope at maximum is zero (No variation) → derivative=0

No variation → derivative=0

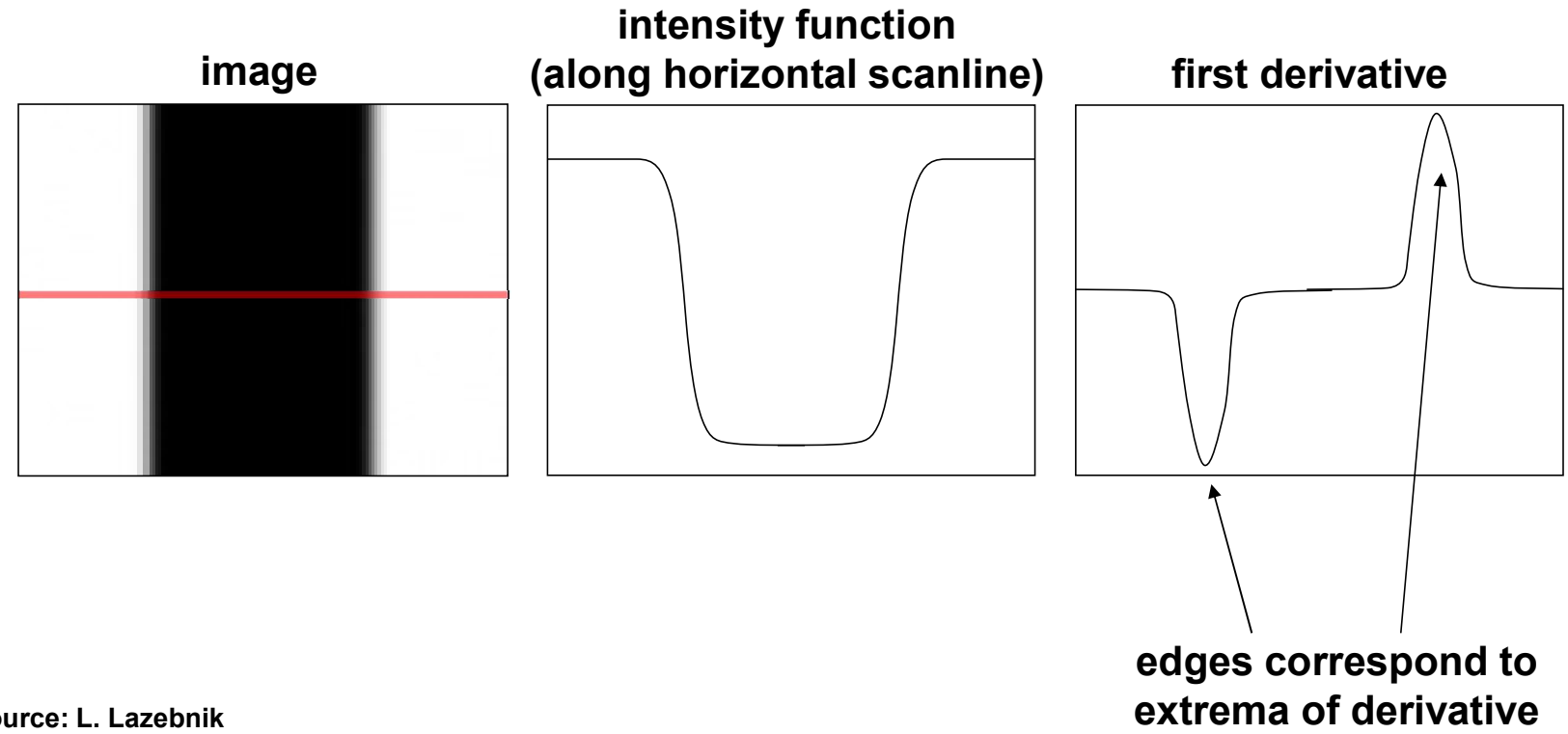
Decreasing positive slope

Increasing negative slope

Let's pay attention to the **blue** line (first derivative of the **red** one).
This will help us to better understand the next slides.

Characterizing edges

- **An edge is a place of *rapid change* in the image intensity function**



Source: L. Lazebnik

Image derivatives

For a 2D function, $f(x, y)$, the partial derivative is

$$\frac{\partial f(x,y)}{\partial x} = \lim_{\varepsilon \to 0} \frac{f(x+\varepsilon, y) - f(x,y)}{\varepsilon}$$

For discrete data, we can approximate the derivative using **finite differences**:

$$\frac{\partial f(x,y)}{\partial x} \approx \frac{f(x+1, y) - f(x,y)}{1}$$

We compute derivatives through the **difference between intensities of adjacent pixels**.

Image derivatives

- **How can we differentiate a digital image F[x,y]?**
  - **Take discrete derivative (finite difference)**

$$\frac{\partial f}{\partial x}[x, y] \approx F[x+1, y] - F[x, y]$$

How would you implement this as a linear filter?

$\frac{\partial f}{\partial x}$:

1	-1	

H_x

$\frac{\partial f}{\partial y}$:

	-1	
	1	

H_y

Source: S. Seitz

Image derivatives

- Odd length avoids undesirable shifting of signal

$$[1 \quad \underline{-1}]$$

Forward difference

$$[4 \quad 7 \quad 8 \quad 1 \quad 3] * [1 \quad \underline{-1}] = [3 \quad 1 \quad -7 \quad 2 \quad 0]$$

The position to be replaced appears underlined.

In this example, we replicate the boundary. So, the value outside the image would be 3: -1*3+1*3=0

(Don't forget to flip kernel before convolving)

$$[\underline{1} \quad -1]$$

Backward difference

$$[4 \quad 7 \quad 8 \quad 1 \quad 3] * [\underline{1} \quad -1] = [0 \quad 3 \quad 1 \quad -7 \quad 2]$$

- To avoid undesirable shift, use central difference kernel

$$\tfrac{1}{2}[1 \quad \underline{0} \quad -1]$$

Central difference

Forward	$f'(x) \approx \frac{f(x+h) - f(x)}{h}$
Backward	$f'(x) \approx \frac{f(x) - f(x-h)}{h}$
Central	$f'(x) \approx \frac{f(x+0.5h) - f(x-0.5h)}{h}$

$h = 2$

Example:

$$[4 \quad 7 \quad 8 \quad 1 \quad 3] * \tfrac{1}{2}[1 \quad \underline{0} \quad -1] = \tfrac{1}{2}[3 \quad 4 \quad -6 \quad -5 \quad 2]$$

We don't usually care about this factor.
We just want to detect differences!

Another (separable) view:

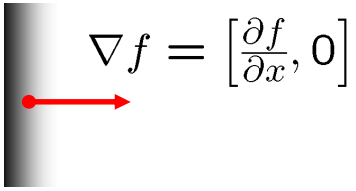
$$\tfrac{1}{2} \cdot [1\ 1] * [1 \quad -1]$$

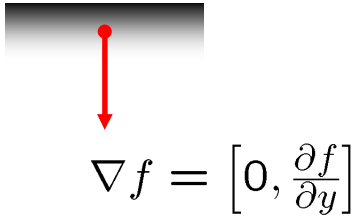
Smoothing Differences

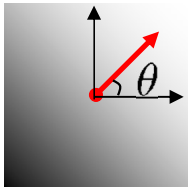
Image gradient

- The *gradient* of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$

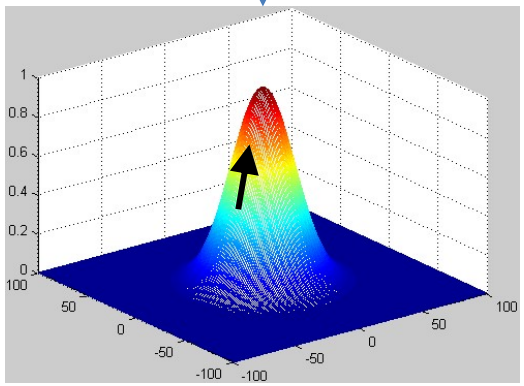
The gradient points in the direction of most rapid increase in intensity







$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$



The *edge strength* is given by the **gradient magnitude**:

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

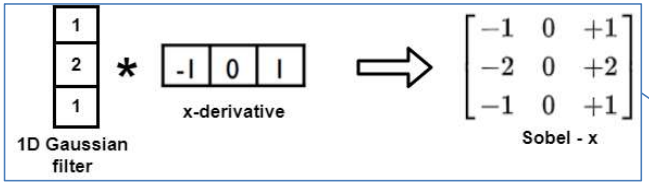
The **gradient direction** is given by:

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

Source: Steve Seitz

Image gradient

Decomposing the Sobel filter



What we are seeing on the first row (synthetic circle): are these correlations or convolutions? 🤔

Horizontal derivative

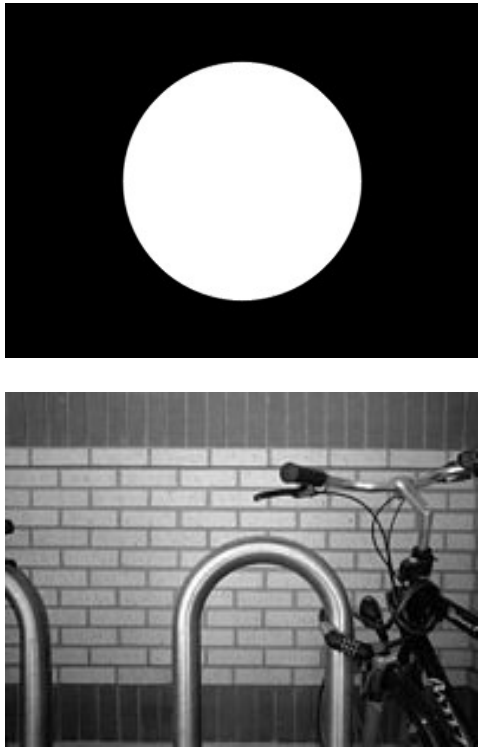
$$\mathbf{G_x} = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

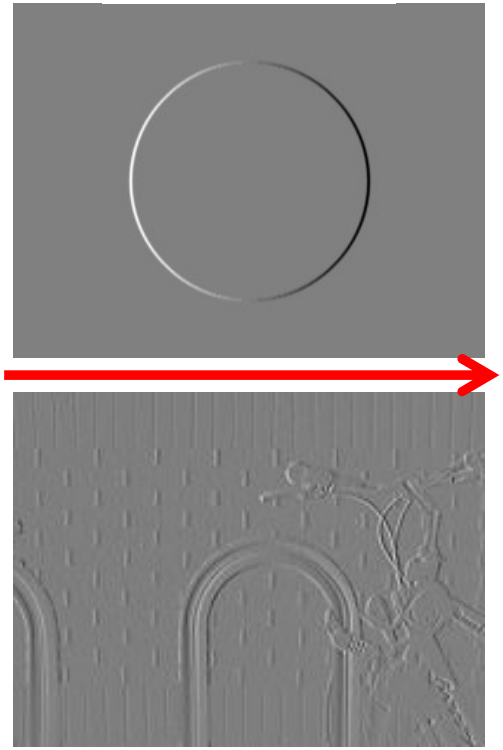
Vertical derivative

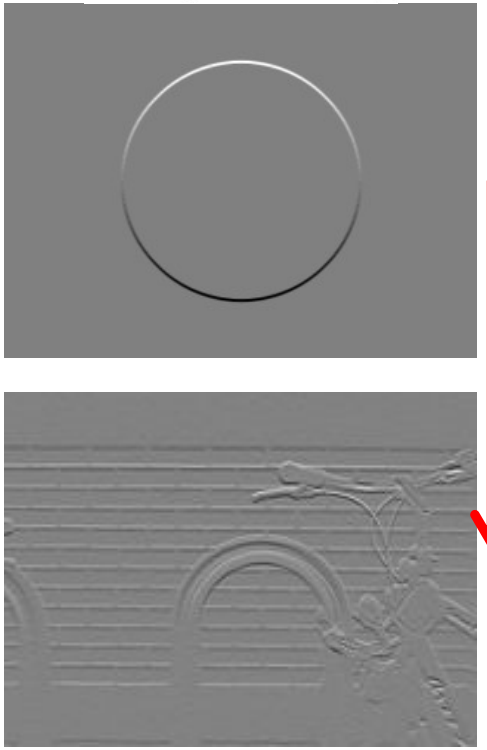
$$\mathbf{G_y} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

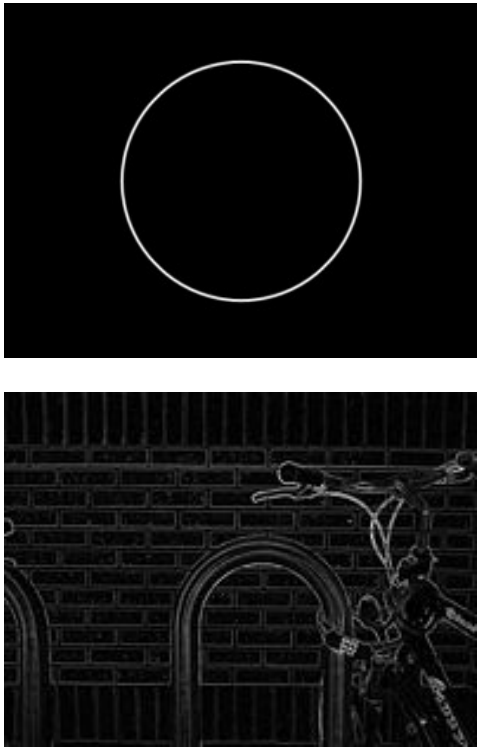
Gradient magnitude

$$\mathbf{G} = \sqrt{\mathbf{G_x}^2 + \mathbf{G_y}^2}$$









Visual examples taken from https://stackoverflow.com/questions/19815732/what-is-the-gradient-orientation-and-gradient-magnitude and https://en.wikipedia.org/wiki/Sobel_operator

Note: These filters give negative values, because derivatives can be negative. Then, for visualization purposes, you have to "transfer" those values to [0,255] or [0,1].

Image gradient

Is there any difference if we apply cross-correlation or convolution with this kernel?? 🤔

Horizontal derivative

$$\mathbf{G_x} = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Vertical derivative

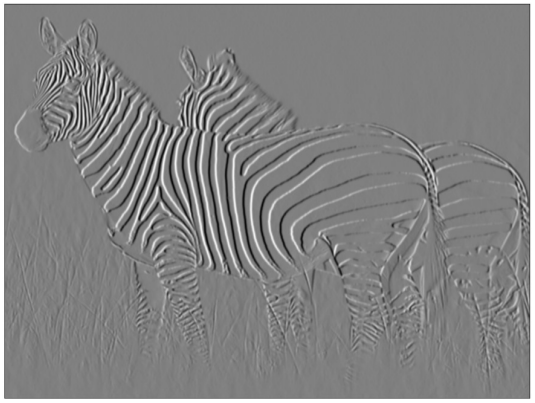
$$\mathbf{G_y} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

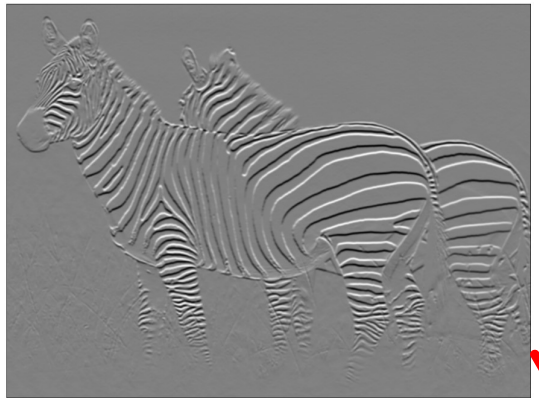
Gradient magnitude

$$\mathbf{G} = \sqrt{\mathbf{G_x}^2 + \mathbf{G_y}^2}$$

Gradient orientation

$$\mathbf{\Theta} = \operatorname{atan2}(\mathbf{G}_y, \mathbf{G}_x)$$

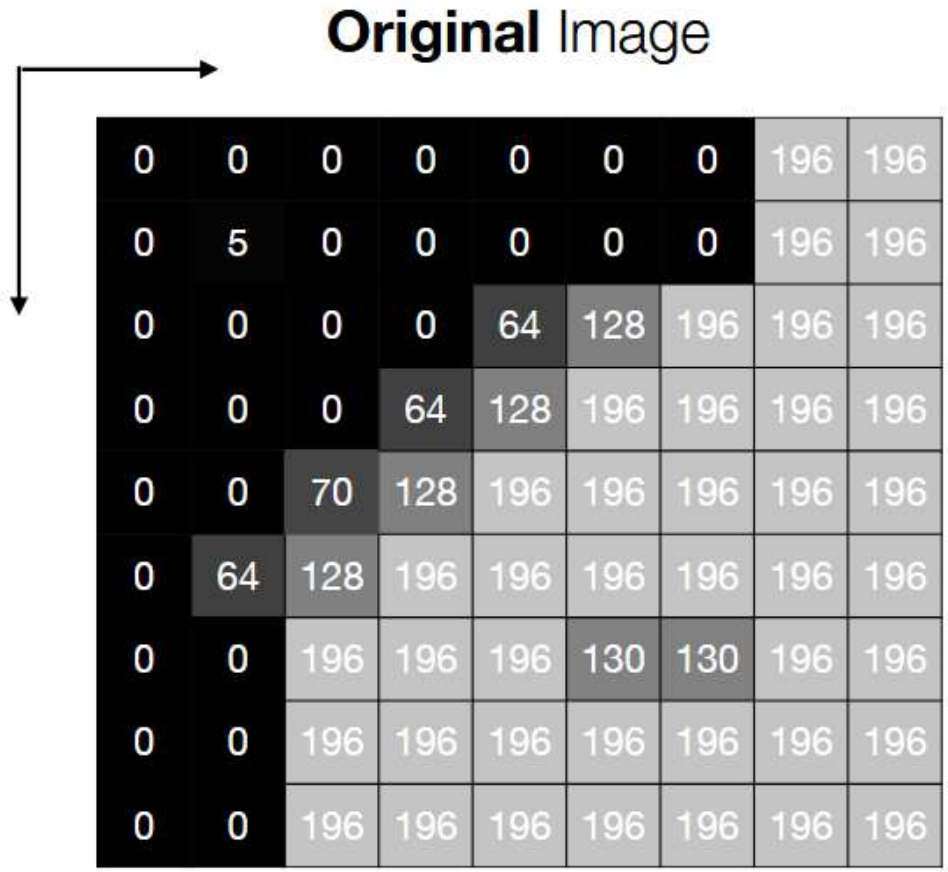








Detailed example of gradient calculation



Source: L. Sigal (https://www.cs.ubc.ca/~lsigal/425_2022W2/Lecture10.pdf)

Detailed example of gradient calculation

Original Image

0	0	0	0	0	0	0	196	196
0	5	0	0	0	0	0	196	196
0	0	0	0	64	128	196	196	196
0	0	0	64	128	196	196	196	196
0	0	70	128	196	196	196	196	196
0	64	128	196	196	196	196	196	196
0	0	196	196	196	130	130	196	196
0	0	196	196	196	196	196	196	196
0	0	196	196	196	196	196	196	196

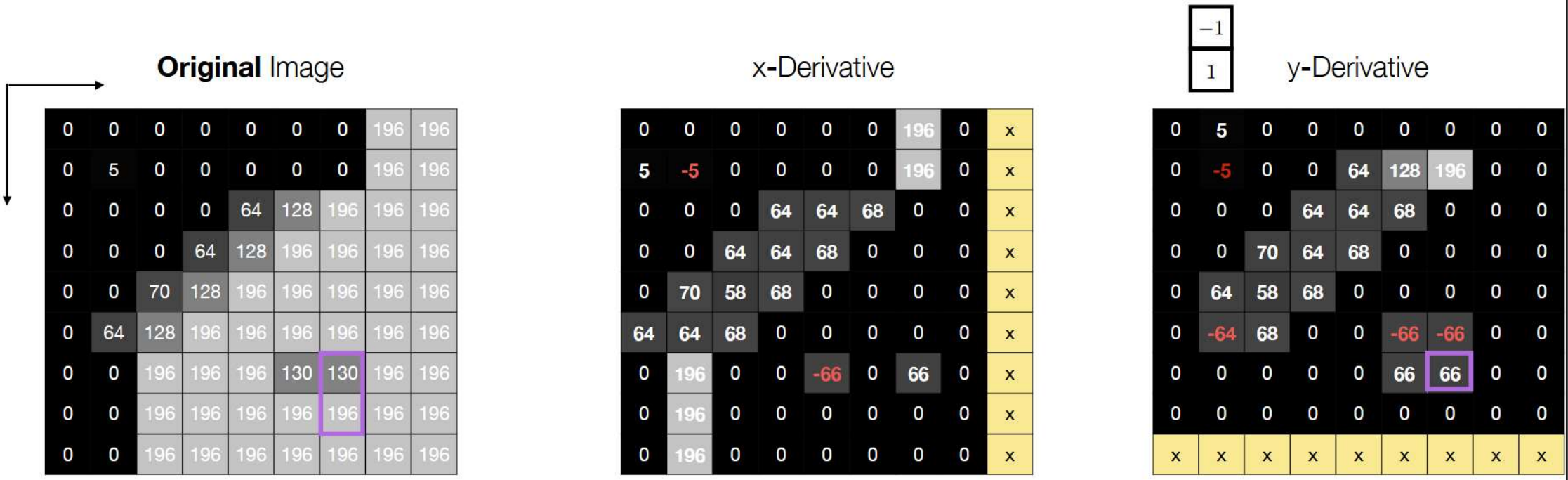
−1	1

x-Derivative

0	0	0	0	0	0	196	0	x
5	-5	0	0	0	0	196	0	x
0	0	0	64	64	68	0	0	x
0	0	64	64	68	0	0	0	x
0	70	58	68	0	0	0	0	x
64	64	68	0	0	0	0	0	x
0	196	0	0	-66	0	66	0	x
0	196	0	0	0	0	0	0	x
0	196	0	0	0	0	0	0	x

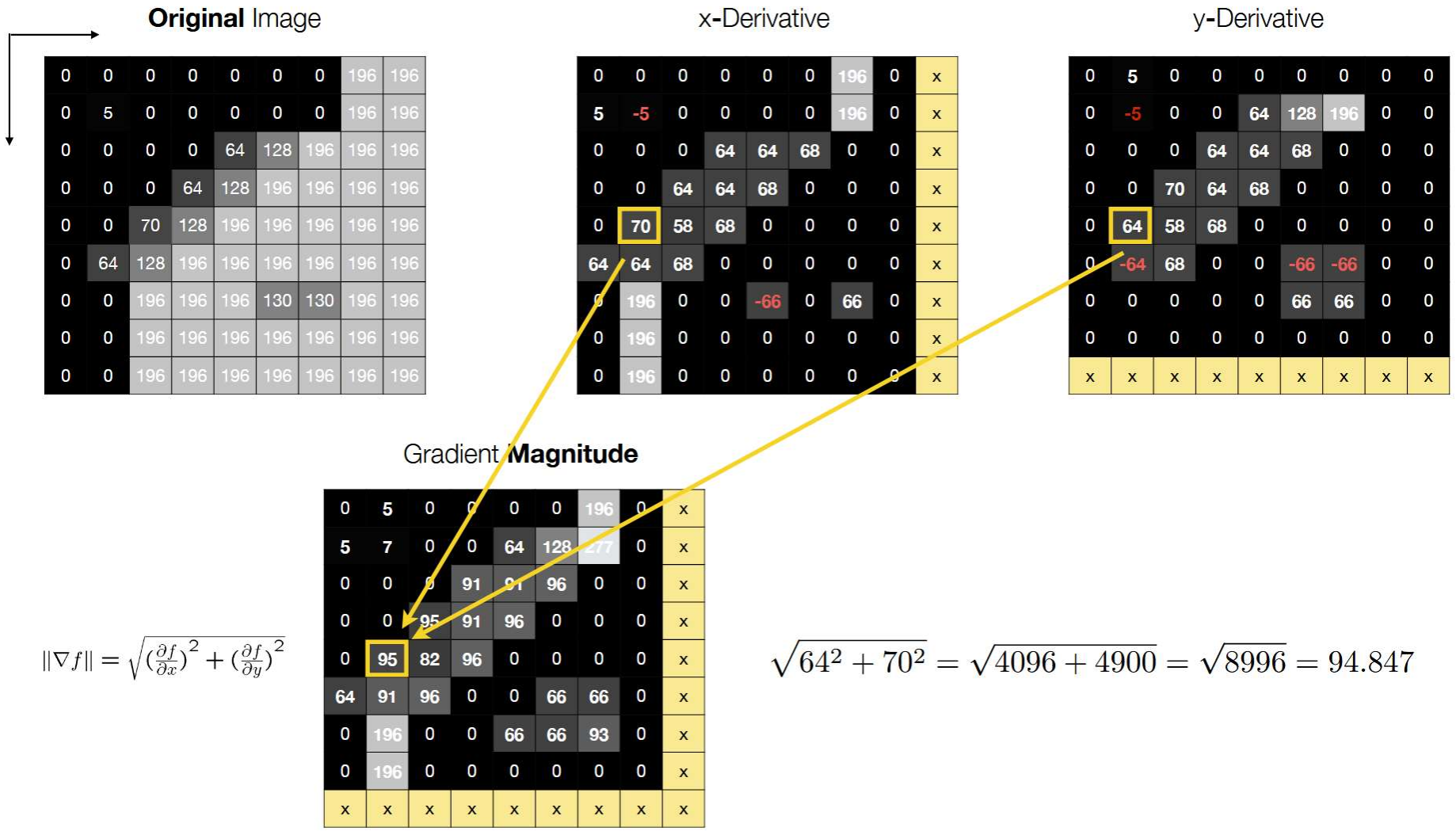
Source: L. Sigal (https://www.cs.ubc.ca/~lsigal/425_2022W2/Lecture10.pdf)

Detailed example of gradient calculation



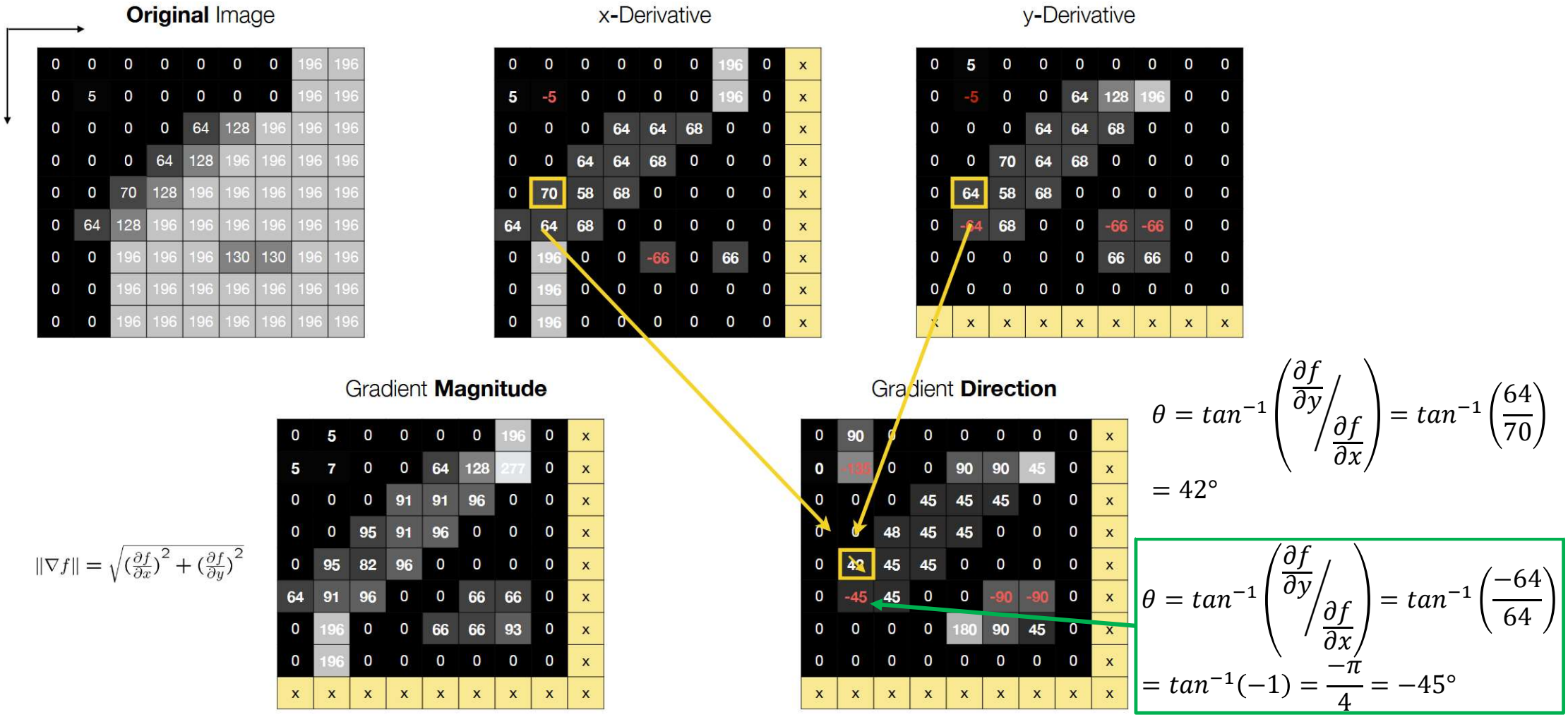
Source: L. Sigal (https://www.cs.ubc.ca/~lsigal/425_2022W2/Lecture10.pdf)

Detailed example of gradient calculation



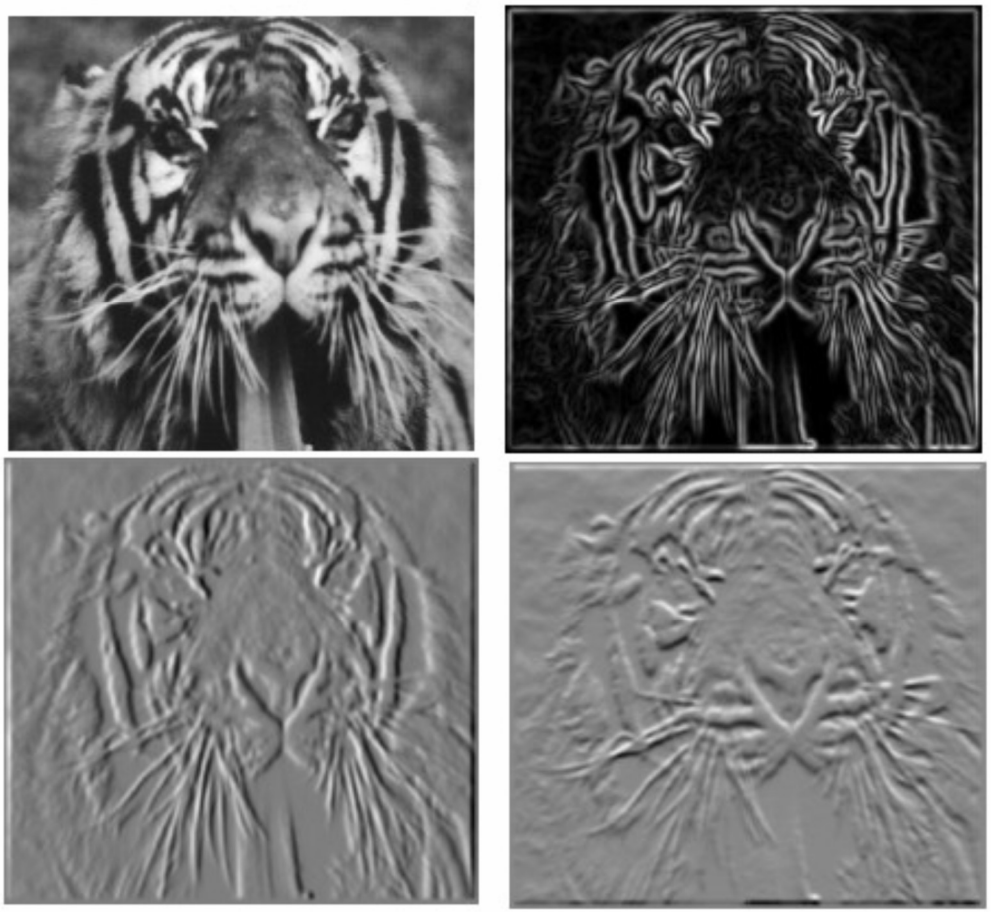
Source: L. Sigal (https://www.cs.ubc.ca/~lsigal/425_2022W2/Lecture10.pdf)

Detailed example of gradient calculation



Source: L. Sigal (https://www.cs.ubc.ca/~lsigal/425_2022W2/Lecture10.pdf)

Image gradient



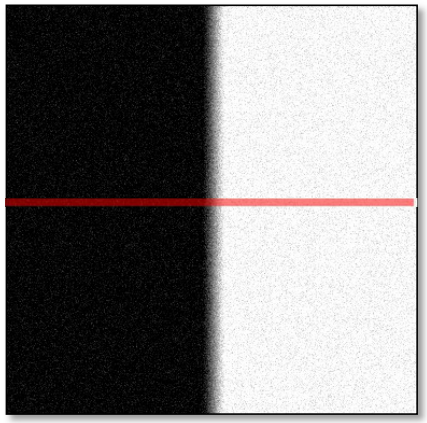
🤔

Which one represents the gradient magnitude?

Which one shows changes with respect to x?

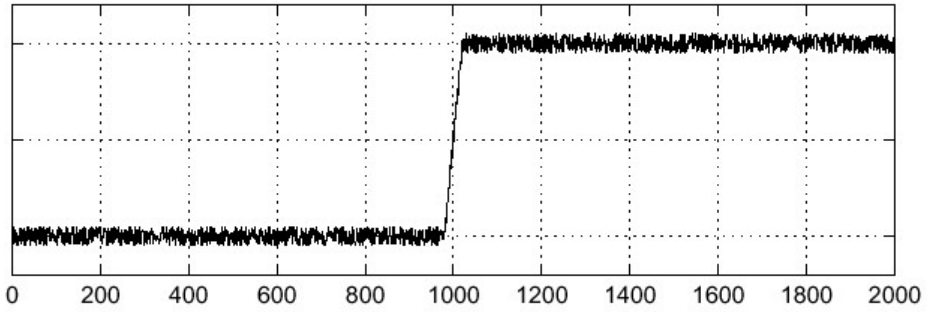
Which one shows changes with respect to y?

Effects of noise

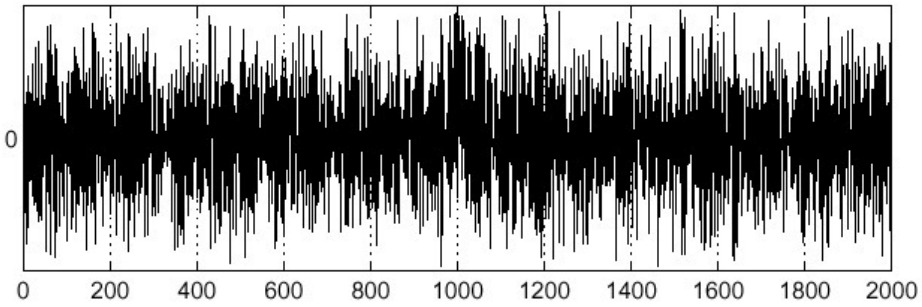


Noisy input image

$f(x)$



$\frac{d}{dx}f(x)$

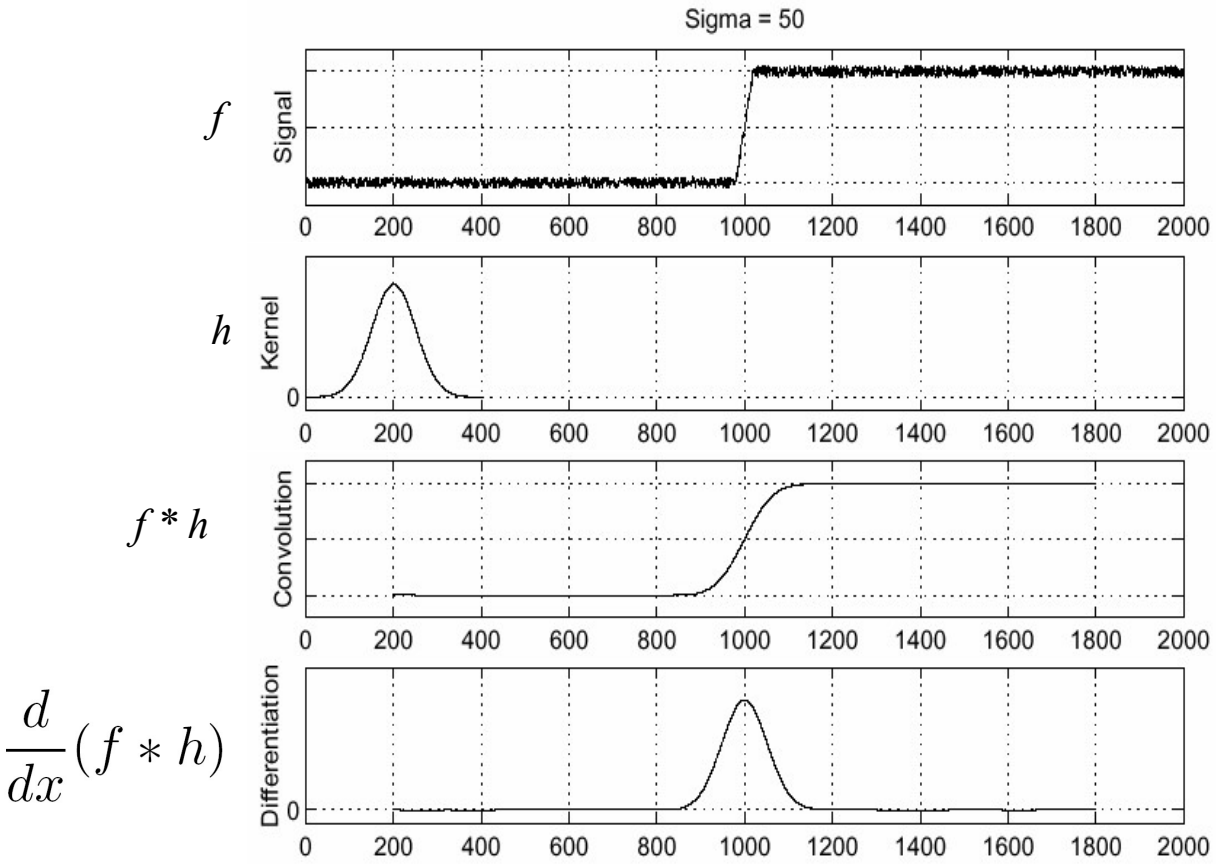


Derivatives amplify noise (high-frequency components)!

Where is the edge?

Source: S. Seitz

Solution: smooth first



How many convolutions do we have here?? 🤔

To find edges, look for peaks $\frac{d}{dx}(f * h)$

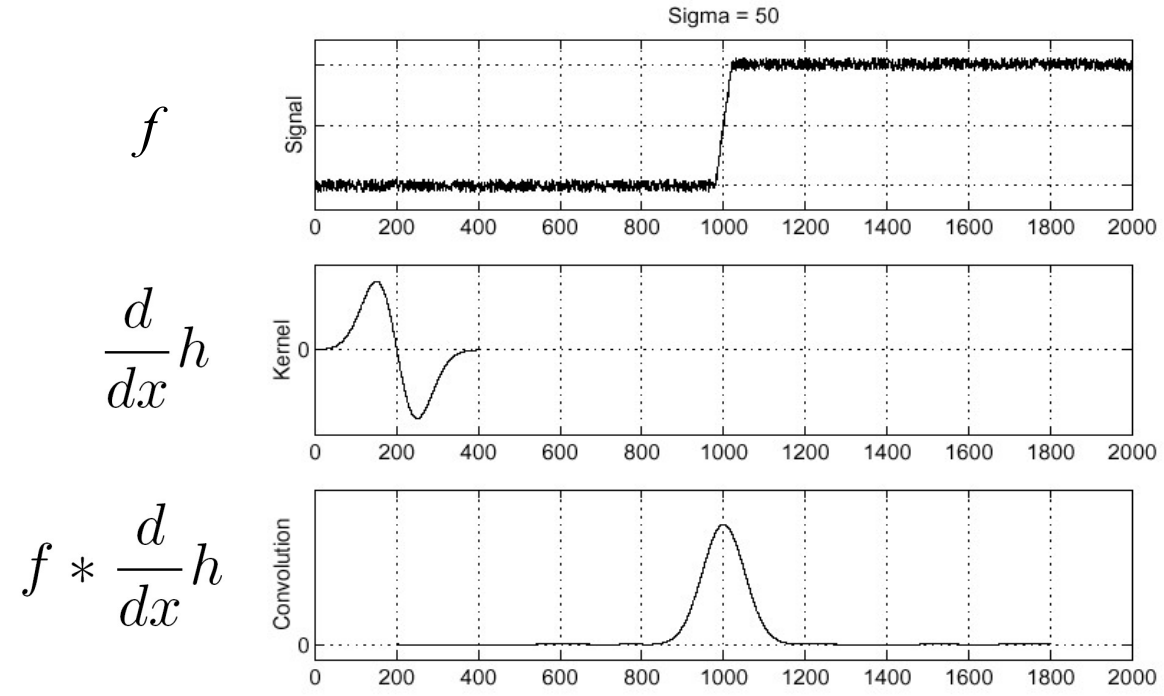
Source: S. Seitz

Associative property of convolution

- Differentiation is convolution, and convolution is associative:

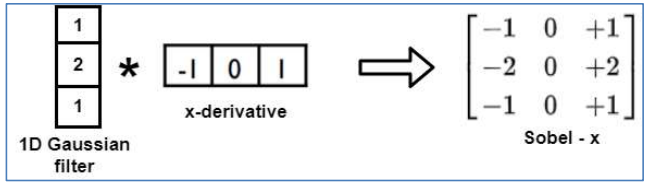
$$\frac{d}{dx}(f * h) = f * \frac{d}{dx}h$$

- This saves us one operation:



Computing derivatives many times implies smoothing (in one direction) and differentiating (in the other)

Examples:



$$\frac{1}{2}\begin{bmatrix}1 & 1\end{bmatrix} * \begin{bmatrix}1 & -1\end{bmatrix} = \frac{1}{2}\begin{bmatrix}1 & 0 & -1\end{bmatrix}$$

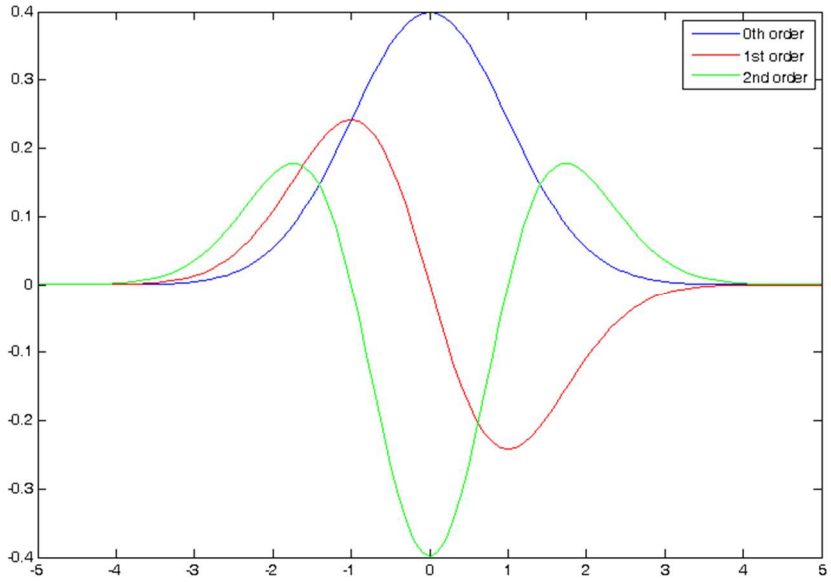
Source: S. Seitz

The 1D Gaussian and its derivatives

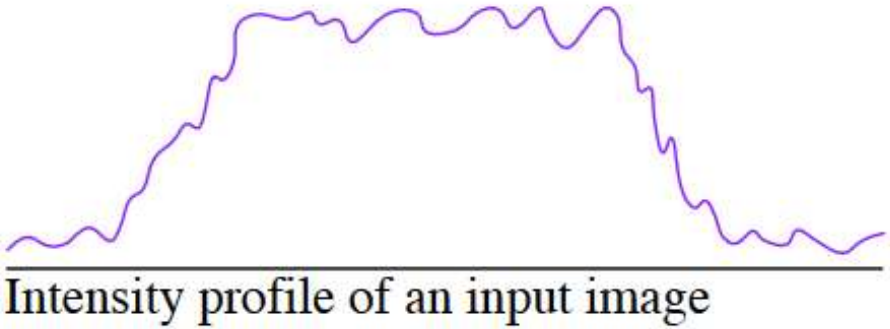
$$g_\sigma(x) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{x^2}{2\sigma^2}\right)$$

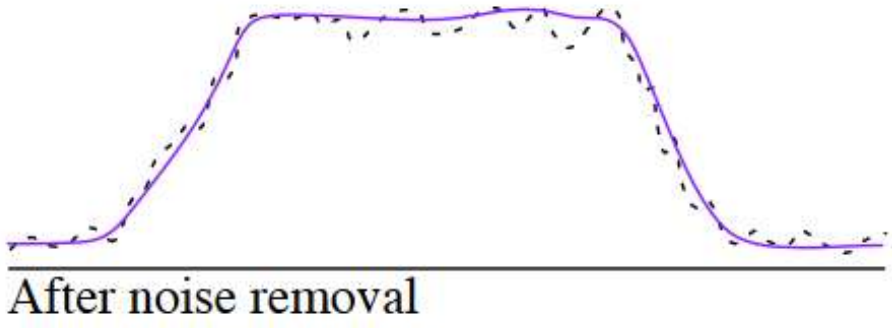
$$g'_\sigma(x) = \frac{\partial g_\sigma(x)}{\partial x} = -\frac{x}{\sigma^2}\frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{x^2}{2\sigma^2}\right) = -\frac{x}{\sigma^2}g_\sigma(x)$$

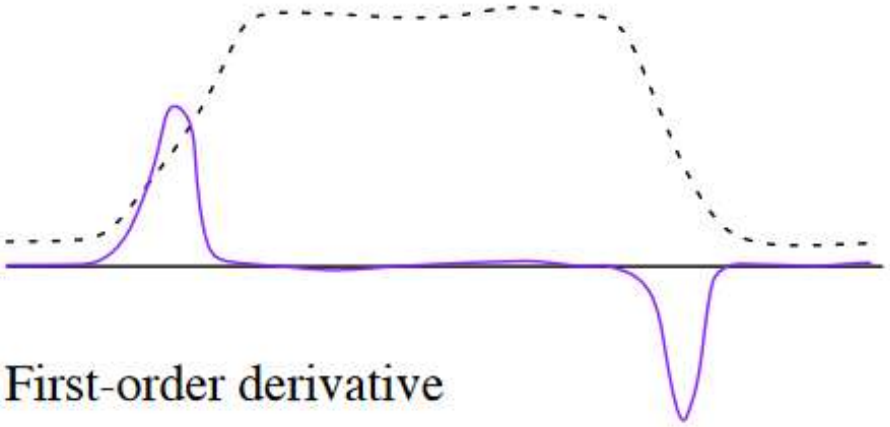
$$g''_\sigma(x) = \frac{\partial^2 g_\sigma(x)}{\partial x^2} = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right)\frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{x^2}{2\sigma^2}\right) = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right)g_\sigma(x)$$

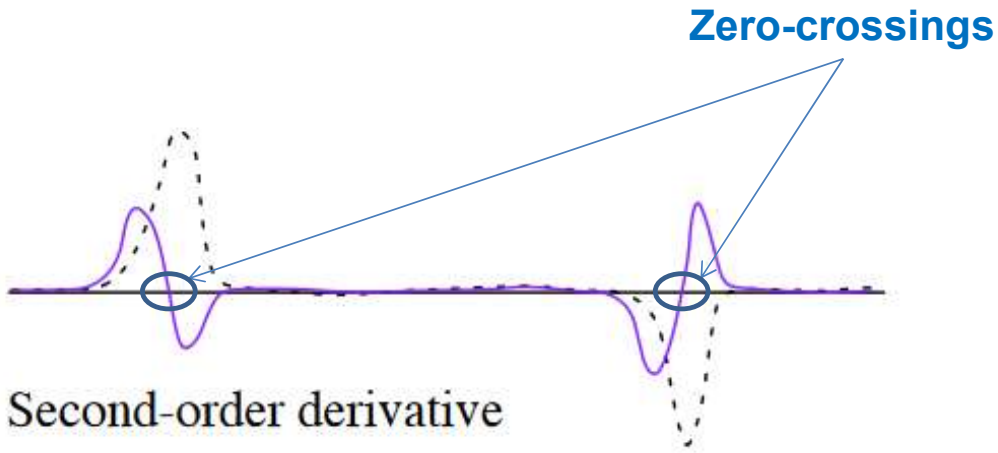


Edge detection using first- and second-order derivatives









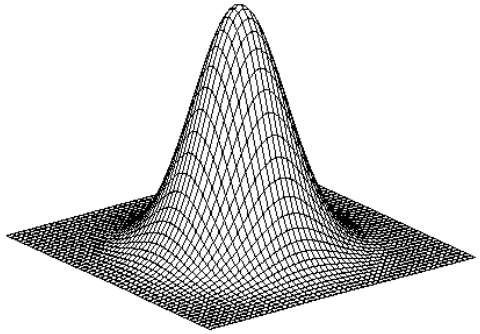
Main advantage of second derivatives?

They provide a **convenient way to localize edges through zero-crossings**. But be careful with noise!

Zero-crossings provide a **simple criterion for edge localization** (see https://www.cs.toronto.edu/~mangas/teaching/320/slides/CSC320L05.pdf)

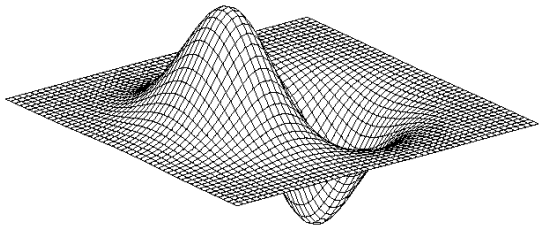
https://www.cs.auckland.ac.nz/~rklette/TeachAuckland.html/775/pdfs/D06-LocalOp1.pdf

2D edge detection filters



Gaussian

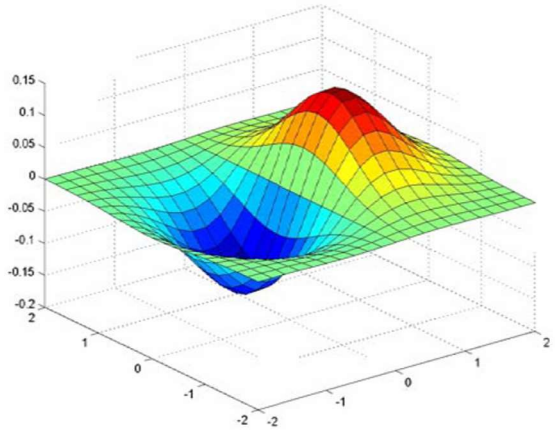
$$h_\sigma(u,v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



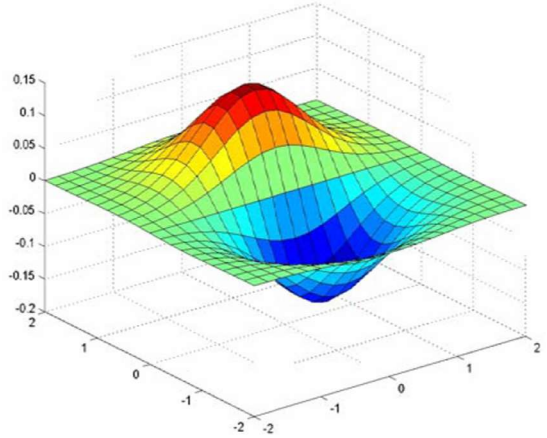
derivative of Gaussian (x)

$$\frac{\partial}{\partial x} h_\sigma(u,v)$$

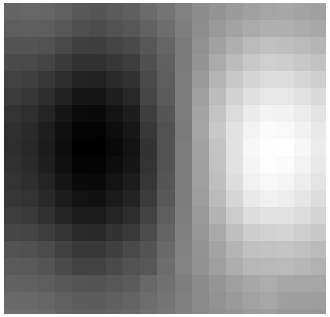
Derivative of Gaussian filter

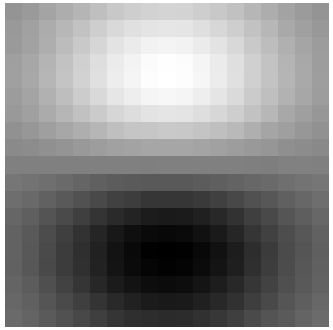


***x*-direction**



***y*-direction**

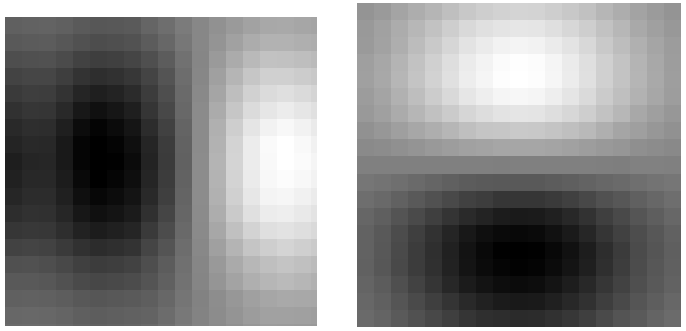




Derivative of Gaussian filter

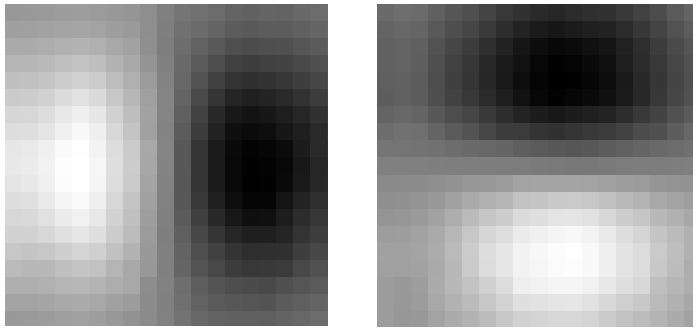
Is the sign really important?? 🤔

***x*-direction** ***y*-direction**



vs

***x*-direction** ***y*-direction**



Common 2D Gaussian derivative kernels

Prewitt

$$Prewitt_x = \frac{1}{3}\begin{bmatrix}1\\1\\1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1 & 0 & -1\end{bmatrix} = \frac{1}{6}\begin{bmatrix}1 & 0 & -1\\1 & 0 & -1\\1 & 0 & -1\end{bmatrix}$$

$$Prewitt_y = \frac{1}{3}\begin{bmatrix}1 & 1 & 1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1\\0\\-1\end{bmatrix} = \frac{1}{6}\begin{bmatrix}1 & 1 & 1\\0 & 0 & 0\\-1 & -1 & -1\end{bmatrix}$$

Sobel

$$Sobel_x = \frac{1}{4}\begin{bmatrix}1\\2\\1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1 & 0 & -1\end{bmatrix} = \frac{1}{8}\begin{bmatrix}1 & 0 & -1\\2 & 0 & -2\\1 & 0 & -1\end{bmatrix}$$

$$Sobel_y = \frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1\\0\\-1\end{bmatrix} = \frac{1}{8}\begin{bmatrix}1 & 2 & 1\\0 & 0 & 0\\-1 & -2 & -1\end{bmatrix}$$

Scharr

$$Scharr_x = \frac{1}{16}\begin{bmatrix}3\\10\\3\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1 & 0 & -1\end{bmatrix} = \frac{1}{32}\begin{bmatrix}3 & 0 & -3\\10 & 0 & -10\\3 & 0 & -3\end{bmatrix}$$

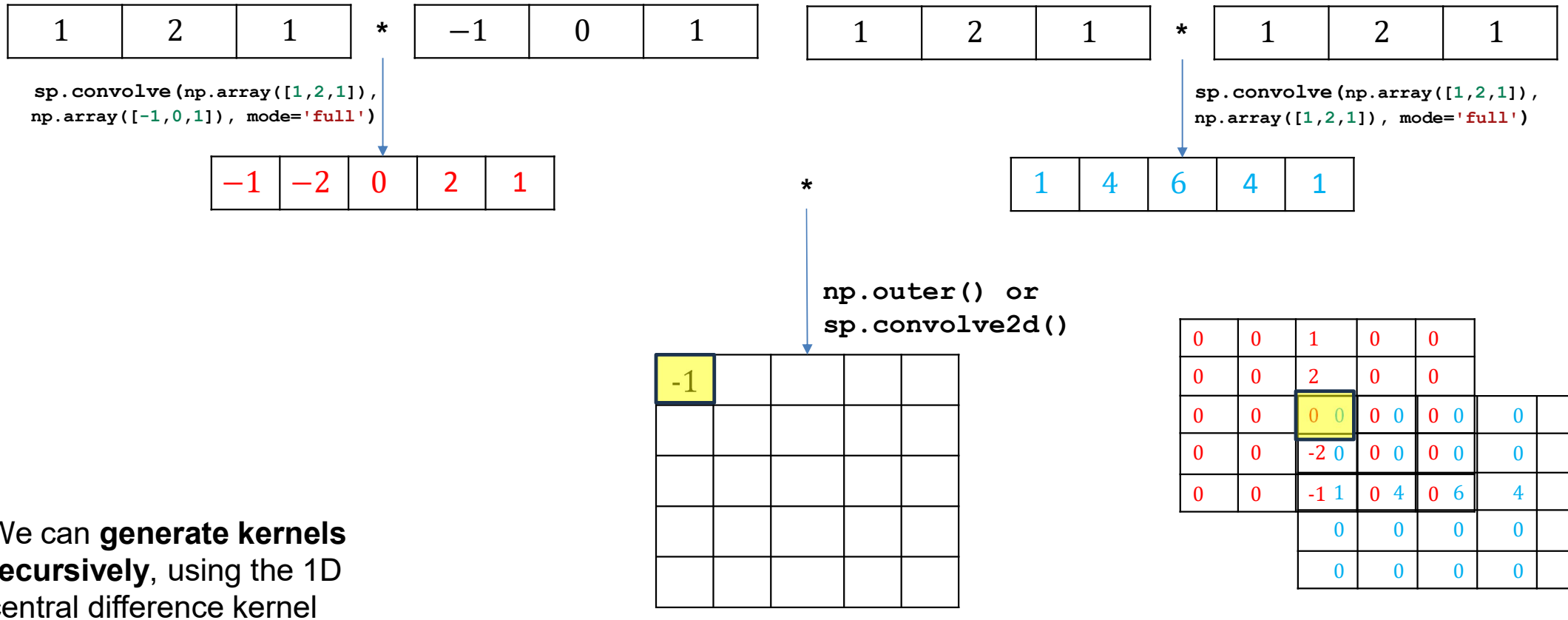
$$Scharr_y = \frac{1}{16}\begin{bmatrix}3 & 10 & 3\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1\\0\\-1\end{bmatrix} = \frac{1}{32}\begin{bmatrix}3 & 10 & 3\\0 & 0 & 0\\-3 & -10 & -3\end{bmatrix}$$

The standard definition of the Sobel operator omits the 1/8 term

- **doesn't make a difference for edge detection**

Creating larger derivative kernels

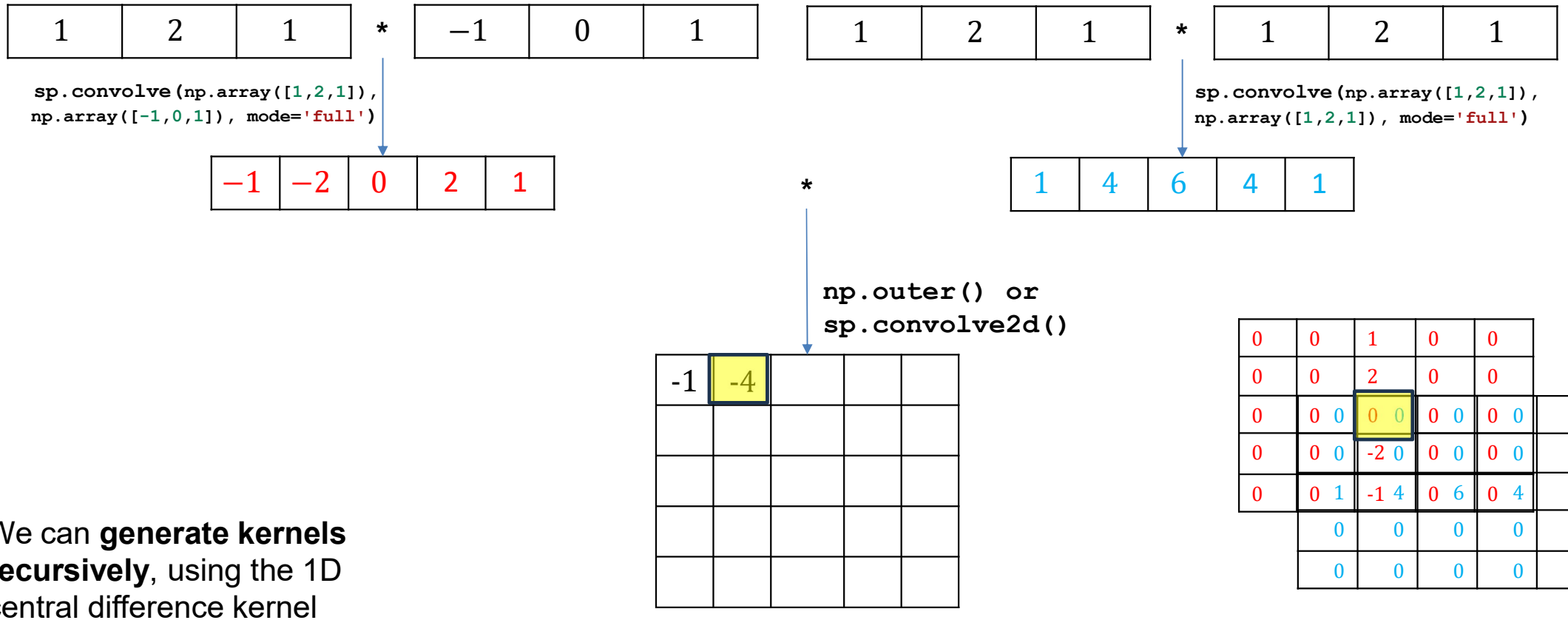
If we want a 5x5 Sobel kernel:



We can **generate kernels recursively**, using the 1D central difference kernel and the binomial!

Creating larger derivative kernels

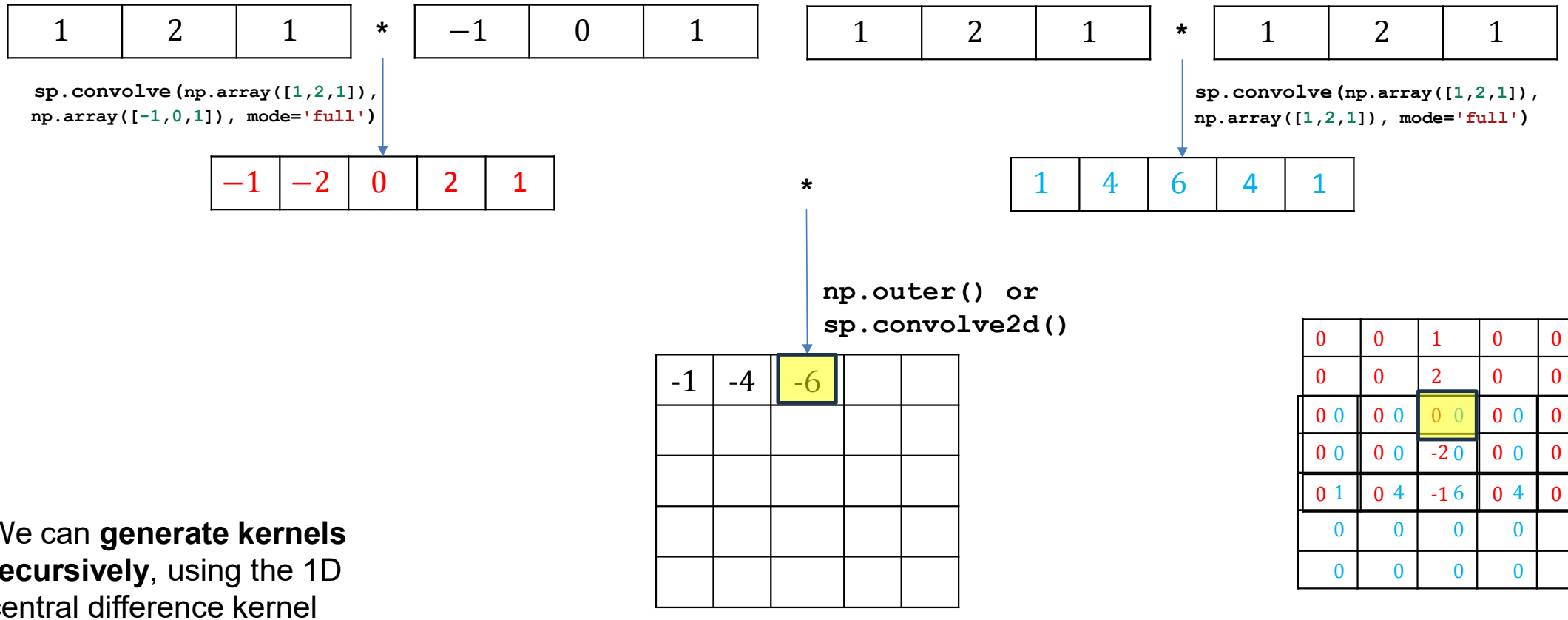
If we want a 5x5 Sobel kernel:



We can **generate kernels recursively**, using the 1D central difference kernel and the binomial!

Creating larger derivative kernels

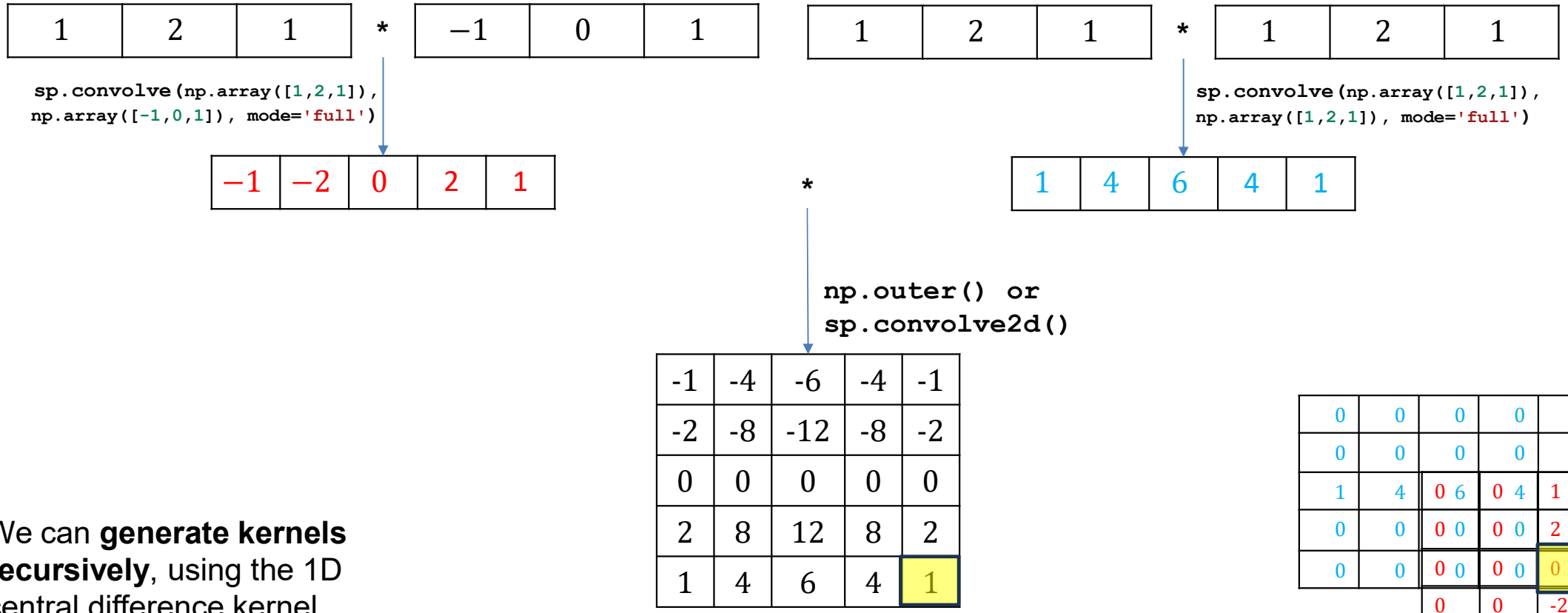
If we want a 5x5 Sobel kernel:



We can **generate kernels recursively**, using the 1D central difference kernel and the binomial!

Creating larger derivative kernels

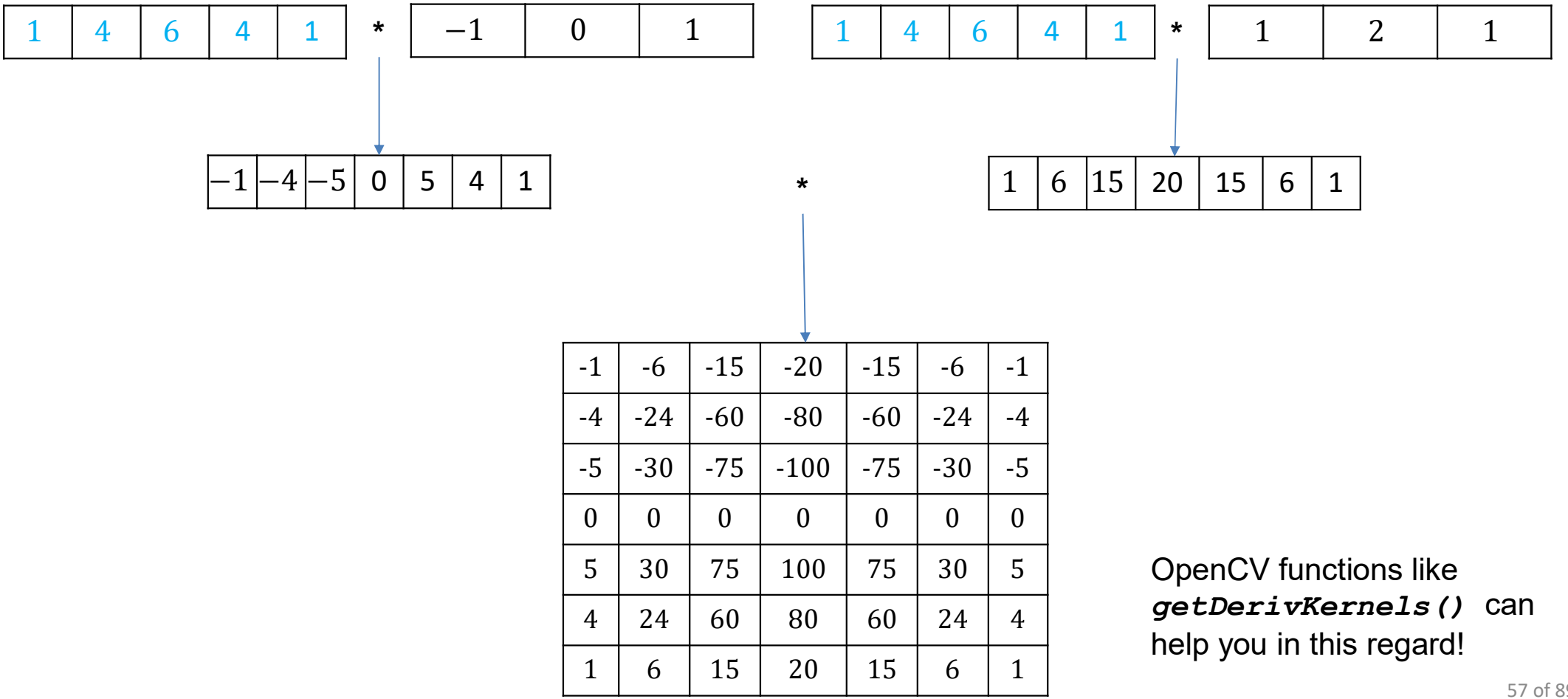
If we want a 5x5 Sobel kernel:



We can **generate kernels recursively**, using the 1D central difference kernel and the binomial!

Creating larger derivative kernels

If we want a 7x7 Sobel kernel:



-1	-6	-15	-20	-15	-6	-1
-4	-24	-60	-80	-60	-24	-4
-5	-30	-75	-100	-75	-30	-5
0	0	0	0	0	0	0
5	30	75	100	75	30	5
4	24	60	80	60	24	4
1	6	15	20	15	6	1

OpenCV functions like ***`getDerivKernels()`*** can help you in this regard!

Separability

First derivative of 2D Gaussian is separable:

$$
\begin{aligned}
G(x,y) &= \frac{1}{2\pi\sigma^2}\exp\left(\frac{-(x^2+y^2)}{2\sigma^2}\right) \\
&= \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{x^2}{2\sigma^2}\right)\cdot\frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G_h(x)\cdot G_v(y) \\
\frac{\partial G(x,y)}{\partial x} &= -\frac{x}{\sigma^2}\cdot\frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2}{2\sigma^2}\right)\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G'_h(x)\cdot G_v(y)
\end{aligned}
$$

horizontal 1D Gaussian derivative kernel

vertical 1D Gaussian kernel

smooth in one direction, differentiate in the other

This allows us to know **which 1D kernels to apply, and in what order, to calculate the derivatives of an image.** In the previous example, the first derivative on X.

Separability

First derivative of 2D Gaussian is separable:

$$
\begin{aligned}
G(x,y) &= \frac{1}{2\pi\sigma^2}\exp\left(\frac{-(x^2+y^2)}{2\sigma^2}\right) \\
&= \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{x^2}{2\sigma^2}\right)\cdot\frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G_h(x)\cdot G_v(y) \\
\frac{\partial G(x,y)}{\partial x} &= -\frac{x}{\sigma^2}\cdot\frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2}{2\sigma^2}\right)\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G'_h(x)\cdot G_v(y)
\end{aligned}
$$

A 2D convolution with a separable kernel can be replaced by two consecutive 1D convolutions.

Separability

1. We know that the 1st derivative of a 2D Gaussian is separable.

$$\begin{aligned}\frac{\partial G(x,y)}{\partial x} &= -\frac{x}{\sigma^2}\cdot\frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2}{2\sigma^2}\right)\exp\left(-\frac{y^2}{2\sigma^2}\right)\\ &= G'_h(x)\cdot G_v(y)\end{aligned}$$

The partial derivative of a Gaussian is composed of the derivative in one direction and the Gaussian in the other.

2. From a practical point of view, we are interested in implicitly **smoothing the signal to avoid magnifying the noise**. So, we smooth in one direction and differentiate in the other.

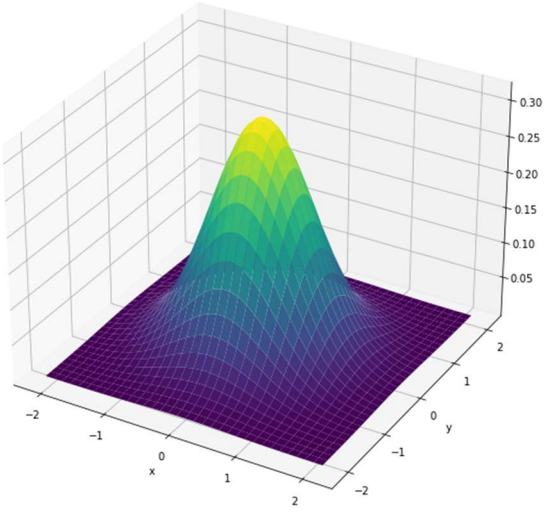
3. Mathematically, **a separable kernel function leads to a separable convolution**. See http://www.songho.ca/dsp/convolution/convolution2d_separable.html

$$\begin{aligned}y[m,n] = (h_1[m]\cdot h_2[n]) * x[m,n] &= h_2[n] * (h_1[m] * x[m,n])\\ &= h_1[m] * (h_2[n] * x[m,n])\end{aligned}$$

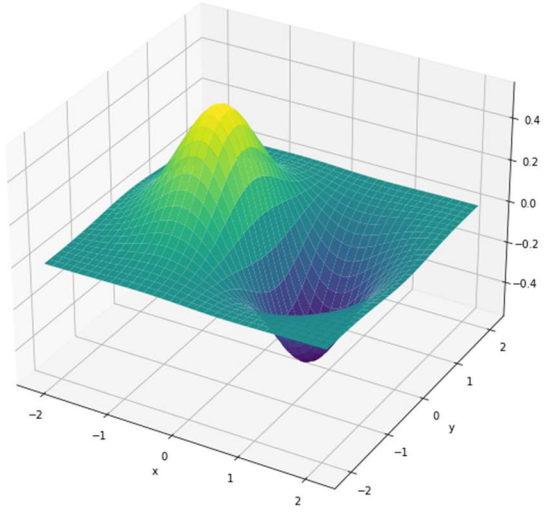
Laplacian of Gaussian

- How many 2nd derivative filters do we have? There are four 2nd partial derivative filters.
- In practice, it's handy to define a single 2nd derivative filter—the Laplacian
  - Instead of using $\frac{\partial^2(G_\sigma(x,y))}{\partial x^2} = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right) G_\sigma(x,y)$ and $\frac{\partial^2(G_\sigma(x,y))}{\partial y^2} = \left(\frac{y^2}{\sigma^4} - \frac{1}{\sigma^2}\right) G_\sigma(x,y)$
  - We employ the Laplacian of Gaussian (LoG) filter

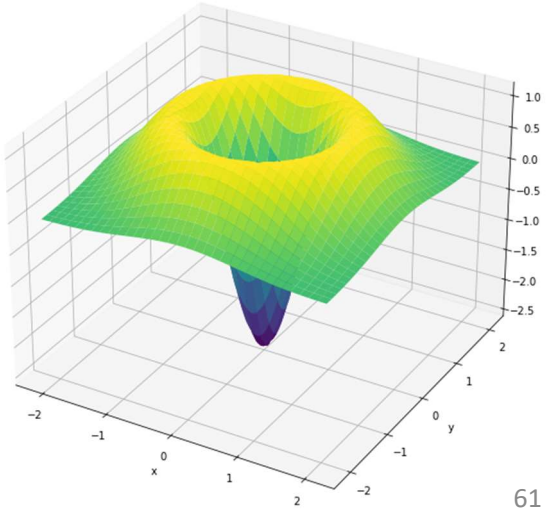
Gaussian



1st derivative of Gaussian (x)



LoG



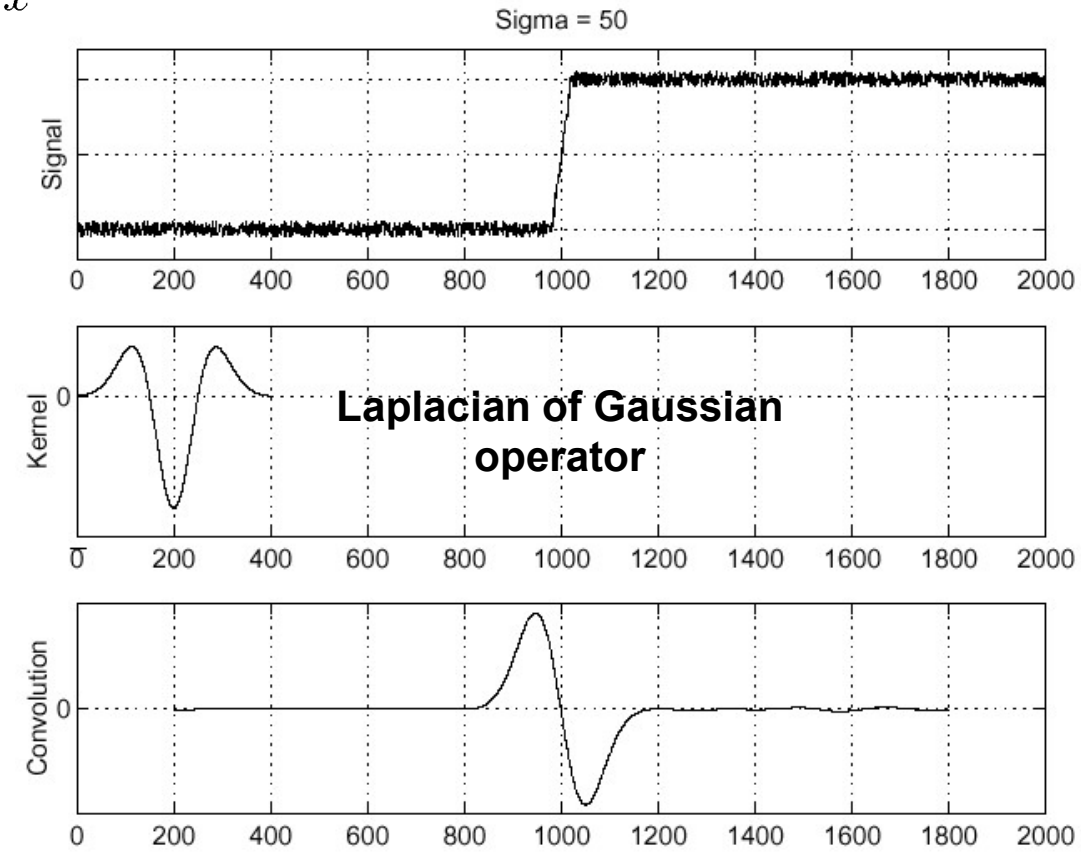
1D Laplacian of Gaussian

Consider $\frac{\partial^2}{\partial x^2}(h \star f)$

f

$\frac{\partial^2}{\partial x^2}h$

$(\frac{\partial^2}{\partial x^2}h) \star f$



Finding these zero-crossings is the goal of the Marr-Hildreth algorithm for edge detection.

Marr, D., & Hildreth, E. (1980). Theory of edge detection. *Procs. of the Royal Society of London*, *207*, 187-217.

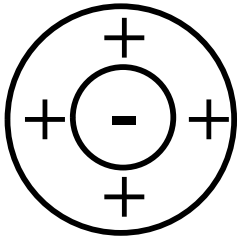
Where is the edge? **Zero-crossings of bottom graph**

Slide credit: Steve Seitz

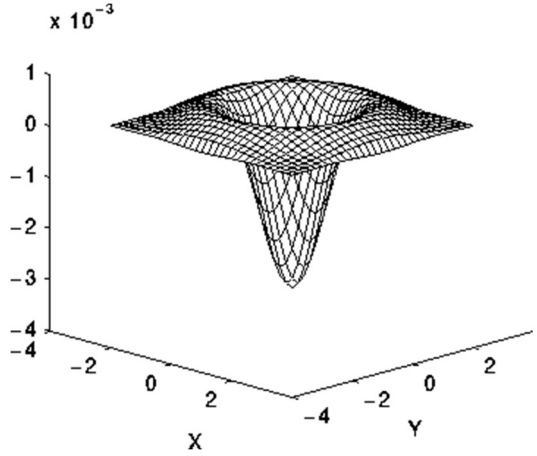
Laplacian of Gaussian

- Example of 3x3 Laplacian kernel:

0	1	0
1	-4	1
0	1	0



"inverted Mexican hat"



- This kernel is not separable, but…
  - we can calculate LoG as the sum of 2D convolutions that are separable
    - Because the Gaussian and its derivatives are!!!
    - In practice, we need 4 1D convolutions (and that is still less computationally expensive than doing a single 2D convolution; unless the kernel is very small)

if the kernel is 7×7 we need 49 multiplications per pixel for the 2D kernel, or 4·7=28 multiplications per pixel for the four 1D kernels; this difference grows as the kernel gets larger

Interesting references about LoG: https://stackoverflow.com/questions/53544983/how-is-laplacian-filter-calculated/53545480#53545480 and https://www.crisluengo.net/archives/1099/

Laplacian of Gaussian

The **Laplacian operator** is the **divergence of the gradient**, and is given by the **sum of the second order derivatives** (i.e. the **trace of the Hessian matrix**):

$$\nabla^2 I = \nabla \cdot \nabla I = \begin{bmatrix} \frac{\partial}{\partial x} & \frac{\partial}{\partial y} \end{bmatrix} \begin{bmatrix} \frac{\partial I}{\partial x} & \frac{\partial I}{\partial y} \end{bmatrix}^T = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

$$\frac{\partial^2 (I * G)}{\partial x^2} + \frac{\partial^2 (I * G)}{\partial y^2}$$

Differentiation property of convolution

$$\frac{\partial^2 (f * g)}{\partial x^2} = f * \frac{\partial^2 g}{\partial x^2}$$

$$\frac{\partial^2 (f * g)}{\partial y^2} = f * \frac{\partial^2 g}{\partial y^2}$$

$$I * \frac{\partial^2 G}{\partial x^2} + I * \frac{\partial^2 G}{\partial y^2}$$

Distributive property of convolution

$$f * (g + h) = f * g + f * h$$

$$I * \left(\frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} \right) = I * \nabla^2 G$$

Laplacian of an image smoothed with a Gaussian kernel

$$\nabla^2 (I * G)$$

Sum of convolutions with the second derivatives of the Gaussian

$$I * \frac{\partial^2 G}{\partial x^2} + I * \frac{\partial^2 G}{\partial y^2}$$

Image convolved with the Laplacian of a Gaussian

$$I * \nabla^2 G$$

Laplacian of Gaussian

How do we compute $I * \frac{\partial^2 G}{\partial x^2} + I * \frac{\partial^2 G}{\partial y^2}$ **?**

We know that Gaussian derivatives are separable:

$$\begin{aligned}\frac{\partial G(x,y)}{\partial x} &= -\frac{x}{\sigma^2} \cdot \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2}{2\sigma^2}\right) \exp\left(-\frac{y^2}{2\sigma^2}\right) \\ &= G'_h(x) \cdot G_v(y)\end{aligned}$$

Then:

$$\frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} = \boxed{G''_h(x) \cdot G_v(y)} + \boxed{G_h(x) \cdot G''_v(y)}$$

2nd derivative in X

2nd derivative in Y

convolution in one direction with the 2nd derivative of the Gaussian, and then in the other direction, convolution with the Gaussian.

convolution in one direction with the Gaussian and then in the other, convolution with the 2nd derivative of the Gaussian.

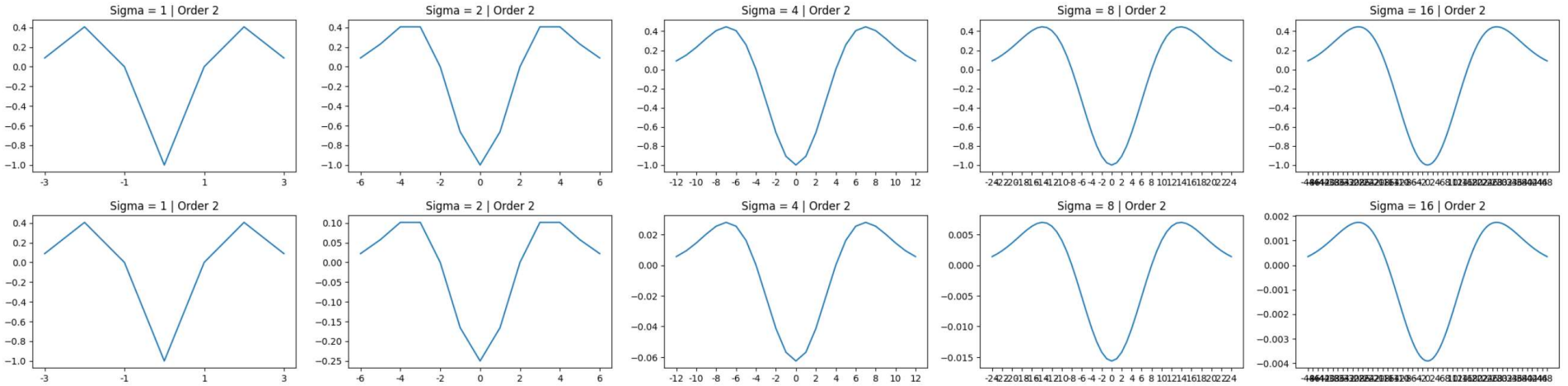
Laplacian of Gaussian

$$L = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right)$$

$$\frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} = G''_h(x) \cdot G_v(y) + G_h(x) \cdot G''_v(y)$$

Why do we multiply by σ^2? 🤔

To **keep response the same (scale-invariant)**, must multiply Gaussian derivative by σ.
Laplacian is the second Gaussian derivative, so it **must be multiplied by σ²**.

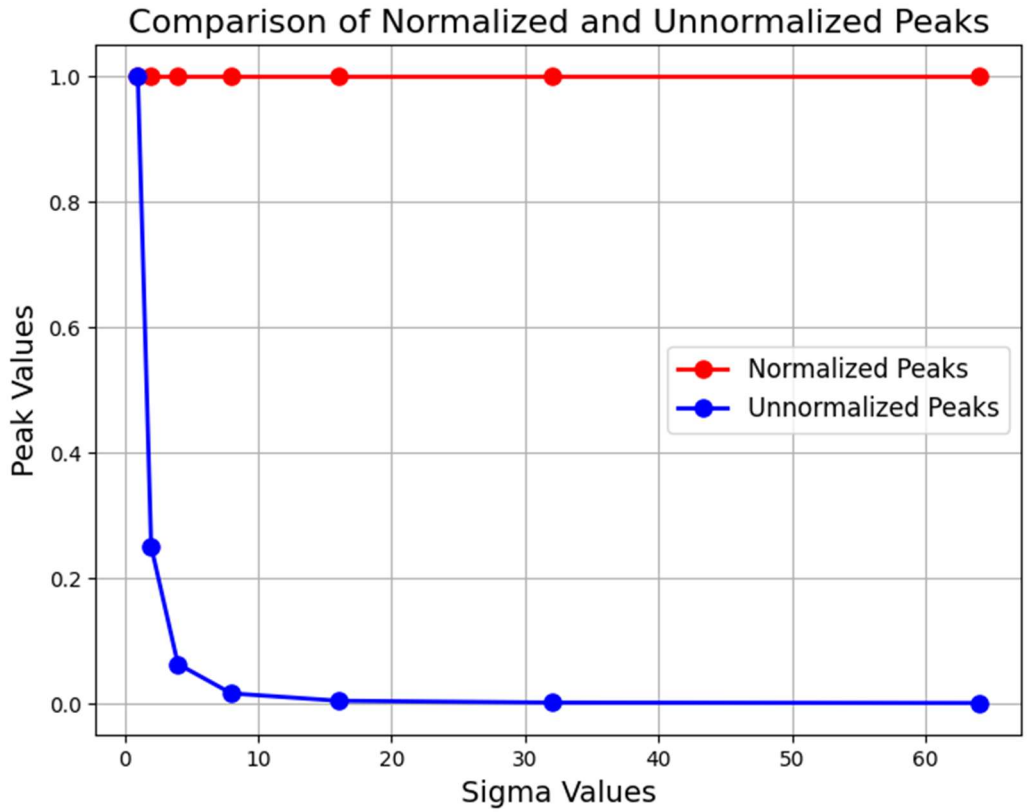


Scale normalized

Scale NO normalized

Laplacian of Gaussian

As σ increases, the peak magnitudes (absolute value) of the first and second derivatives of the Gaussian decrease.

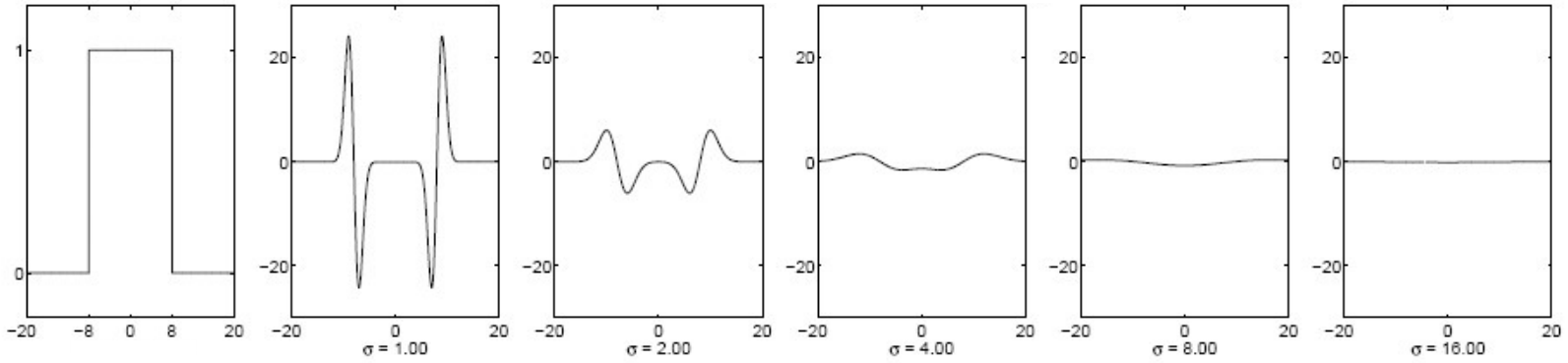


Laplacian of Gaussian

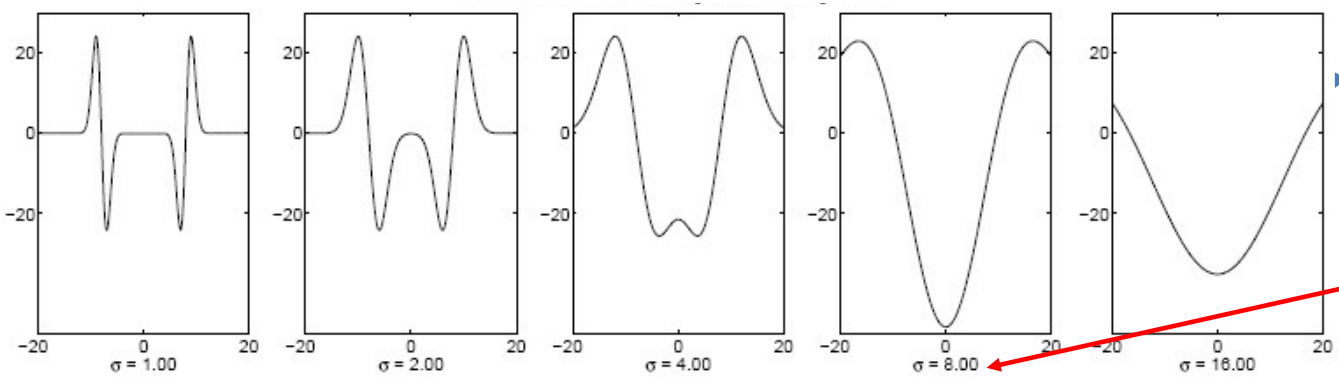
- The response of a derivative of Gaussian filter to a perfect step edge decreases as σ increases
- To keep response the same (scale-invariant), must multiply Gaussian derivative by σ
- Laplacian is the second Gaussian derivative, so it must be multiplied by σ^2

Original signal

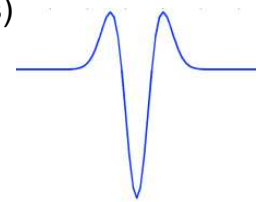
Unnormalized Laplacian response



Scale-normalized Laplacian response



Responses of convolving the original signal with the LoG (using different sigmas)



Maximum response

Laplacian of Gaussian approximation

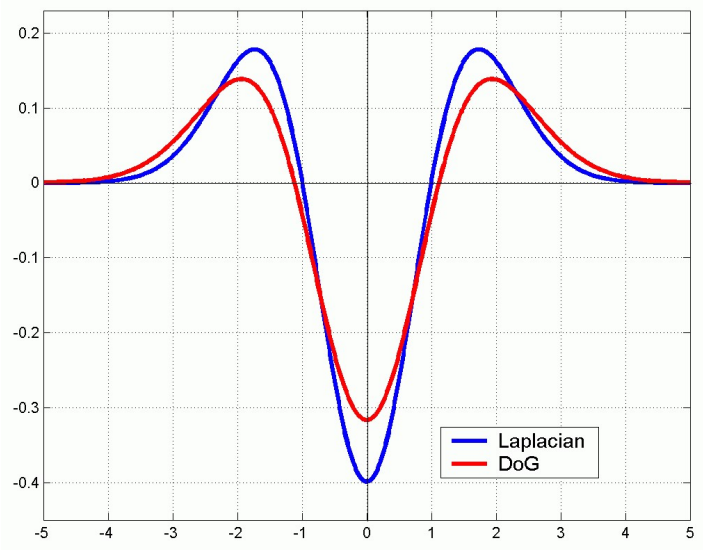
- Approximating the Laplacian with a difference of Gaussians:

$$L = \sigma^2\big(G_{xx}(x,y,\sigma) + G_{yy}(x,y,\sigma)\big)$$

(Laplacian)

$$DoG = G(x,y,k\sigma) - G(x,y,\sigma)$$

(Difference of Gaussians)



Advantage: we don't even need to implement the second derivative. Just compute the difference of blurred images.

$$G(x,y,k\sigma) - G(x,y,\sigma) \approx (k-1)\cdot L$$

Effect of σ on derivatives (scales)





σ = 1



σ = 3

The apparent structures differ depending on Gaussian's scale parameter.

Larger values: larger scale edges detected.
Smaller values: finer features detected.

Summary: Two types of convolution kernels

Smoothing

$$\sum g_i = 1$$

(Smoothing a constant function should not change it)

Example: (1/4) * [1 2 1]

Low-pass filter (Gaussian)

Differentiating

$$\sum g_i = 0$$

(Differentiating a constant function should return 0)

Example: [-1 0 1]

High-pass filter (derivative of Gaussian)

Summary: Two types of convolution kernels

Two closely related problems:



blur
(to remove noise)



differentiate
(to highlight details)

Summary: Two types of convolution kernels

Two closely related problems:





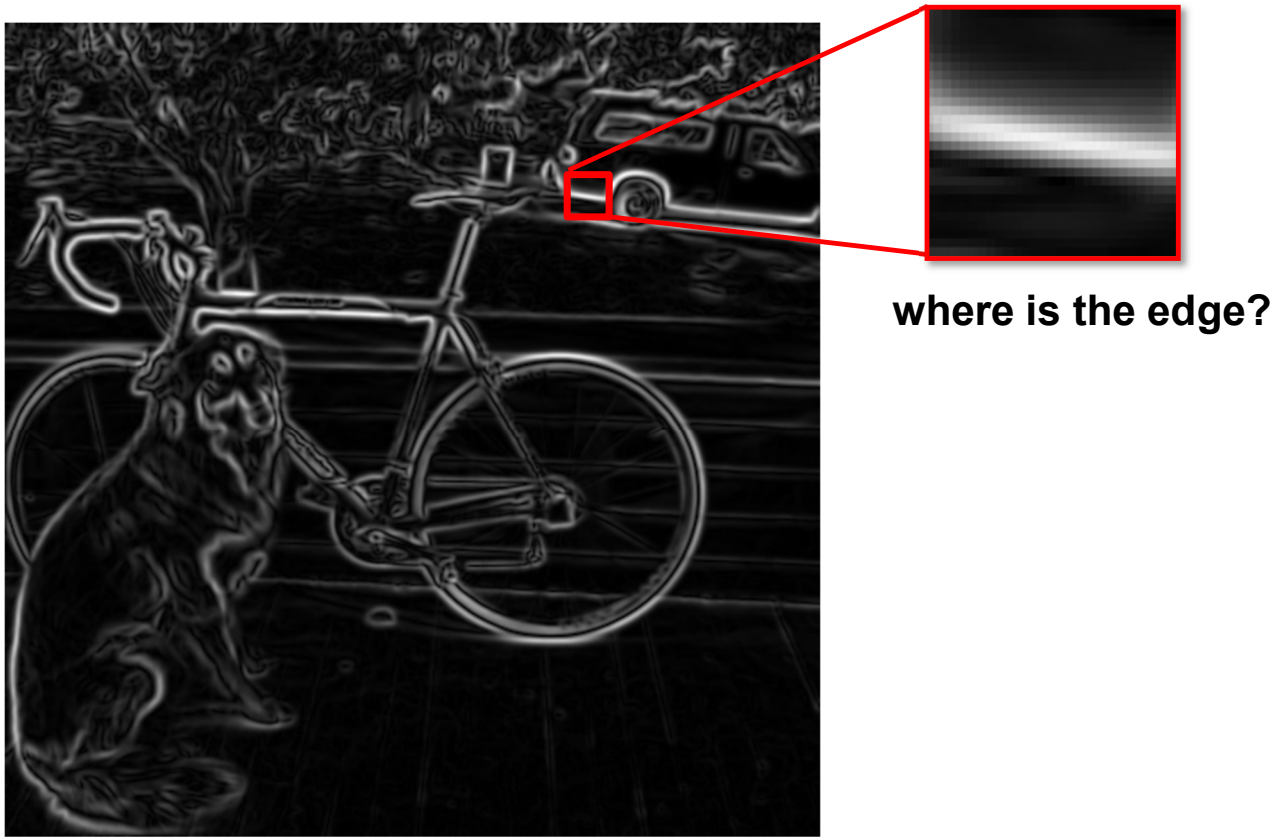
blur
(to remove noise)

differentiate
(to highlight details)

Underlying math is the same!

Canny Edge Detector

- Sometimes we need to accurately locate the boundary….

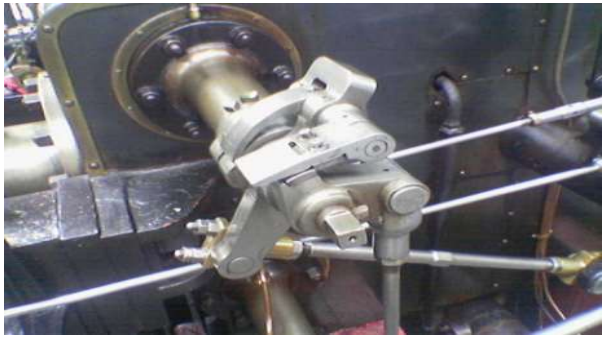


Canny Edge Detector

- Canny, J., "A Computational Approach To Edge Detection", IEEE Trans. on Pattern Analysis and Machine Intelligence, 8(6):679–698, 1986 (~50K citations in Google Scholar)
- Still popular because
  - easy to implement
  - small number of intuitive parameters
  - computationally efficient
  - good results

Python:
https://docs.opencv.org/4.8.0/da/d22/tutorial_py_canny.html

Original Image



Sobel Magnitude

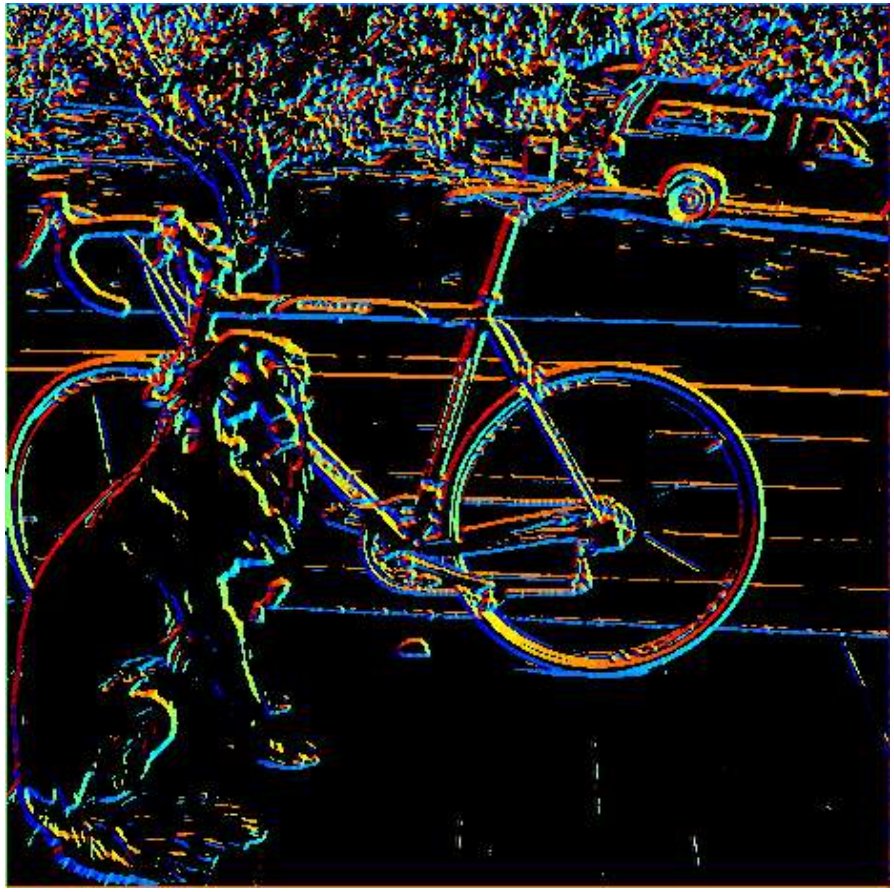


Canny Edge Detector



Canny Edge Detector

1) Filter image with derivative of Gaussian and compute the gradient (magnitude and orientation) at each pixel



theta = atan2(gy, gx)

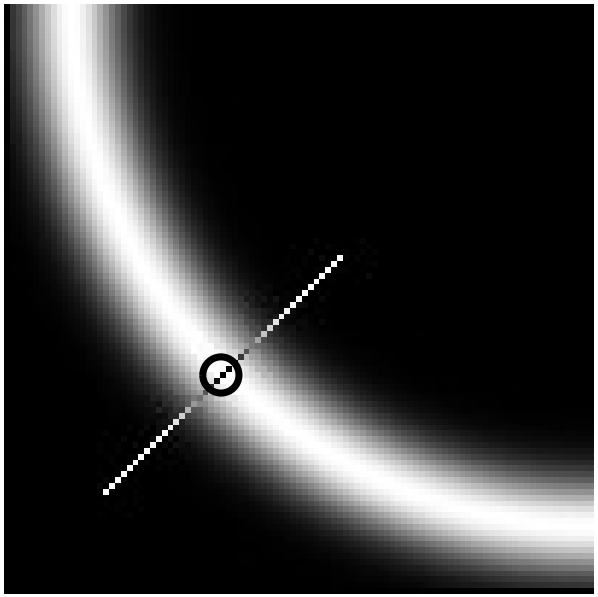


Gradient orientation angle

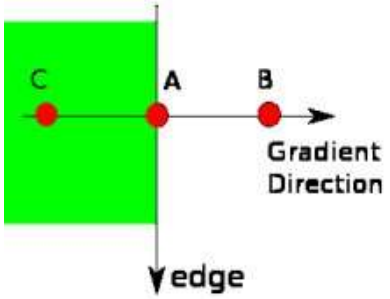
Canny Edge Detector

2) Non-maximum suppression

A full scan of the image is done to remove any unwanted pixels which may not constitute the edge.



For each pixel, we keep it only if it has the highest gradient magnitude along the gradient direction in its local neighborhood.
This makes the edges thinner (1 pixel wide).



Since pixel A has orientation 0º → we check its left (C) and right (B) neighbors

- We check whether each pixel is a **local maximum in its neighborhood along gradient direction**
  - Point A is on the edge. Gradient direction is normal to the edge. Point B and C are in the gradient direction.
  - Point A is checked with point B and C to see if it forms a local maximum.
  - If so, it is considered for next stage, otherwise, it is suppressed (put to zero).

https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

Before Non-max Suppression



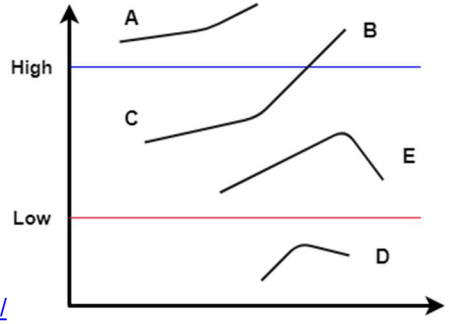
After Non-max Suppression



Canny Edge Detector

3) Linking and thresholding edges (hysteresis)

- We only want strong edges
- 2 thresholds (high, low), 3 cases (R: response):
  - R > high: strong edge!
  - R < low: no edge
  - R < high but R > low: weak edge
    - **Weak edges are edges iff they connect to strong ones**
    - Look in some neighborhood (usually 8 closest)



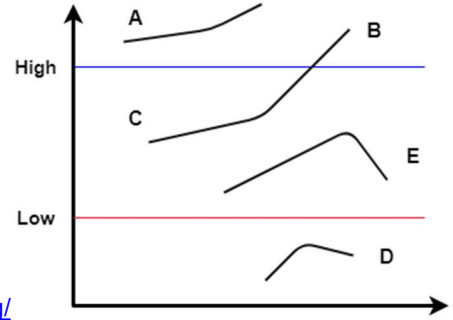


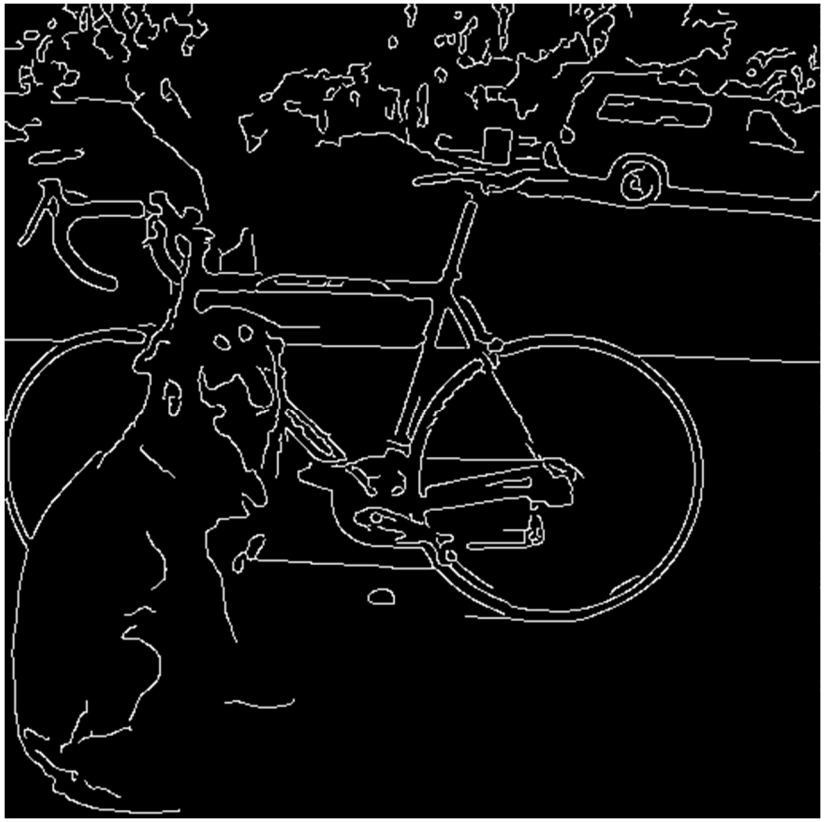
https://theailearner.com/tag/hysteresis-thresholding/

Canny Edge Detector

3) Linking and thresholding edges (hysteresis)

- We only want strong edges
- 2 thresholds (high, low), 3 cases (R: response):
  - R > high: strong edge!
  - R < low: no edge
  - R < high but R > low: weak edge
    - **Weak edges are edges iff they connect to strong ones**
    - Look in some neighborhood (usually 8 closest)





https://theailearner.com/tag/hysteresis-thresholding/

Canny Edge Detector



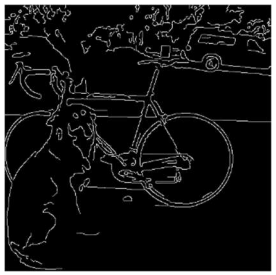
0. Many times we first smooth the image to reduce the impact of noise



1. Filter image with derivative of Gaussian and find magnitude and orientation of gradient



2. Non-maximum suppression



3. Linking and thresholding (hysteresis):
   - Define two thresholds: low and high
   - Use the high threshold to start edge curves and the low threshold to continue them

Source: D. Lowe, L. Fei-Fei, J. Redmon

Canny Edge Detector



original **Canny with** $\sigma_d = 1$ **Canny with** $\sigma_d = 2$

Parameters to tune:

high threshold
low threshold
σ_d: width of the Gaussian derivative filter
σ_s: width of the Gaussian smoothing filter

- The choice of σ_d depends on desired behavior
  - Large σ_d detects "large-scale" edges
  - Small σ_d detects fine edges

Adapted from S. Seitz

Canny Edge Detector



original image



high threshold
(strong edges)



low threshold
(weak edges)



hysteresis threshold

Source: L. Fei-Fei

Homework

Let's imagine a 2D Gaussian kernel of size $9x9$.

- What would the equivalent sigma be (assuming that k has to be the smallest integer greater than or equal to 3σ)?
- If we wanted to apply this same $9x9$ kernel 3 consecutive times, what would the equivalent kernel size be?

  Note: When you apply the same kernel of size N a total of k times, the equivalent filter size is $N + (k-1) \cdot (N-1)$
- If we wanted to apply this same $9x9$ kernel 3 consecutive times, what would the equivalent kernel size be using the property that the Gaussian is closed under convolution (i.e., convolving N consecutive times with sigma is equal to a single convolution with a Gaussian with $\sigma\sqrt{N}$)? In this case, we look for the filter with the equivalent blurring effect.

Homework (exam quizzes, in Spanish)

¿Qué propiedad o teorema relativo a la convolución es el que, principalmente, permite la creación/composición de filtros más complejos a partir de filtros más sencillos?

a) Asociativa
b) Conmutativa
c) Distributiva
d) Teorema de la Convolución

¿Qué tipo de filtro considera que es el siguiente?

$$\begin{matrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{matrix}$$

a) Suavizado Gaussiano
b) Laplaciana de la Gaussiana
c) Sharpening
d) Ninguna de las anteriores

Homework (exam quizzes, in Spanish)

Si suavizamos una imagen tres veces consecutivas con un filtro Gaussiano σ, ¿cuál sería el sigma equivalente que nos permitiría obtener el mismo resultado suavizando una única vez?

a) $\sigma\sqrt{3}$
b) $\sqrt{3\sigma}$
c) $3\sqrt{\sigma}$
d) Ninguna de las anteriores

¿Cuál de las siguientes afirmaciones sobre los algoritmos de Canny y Otsu es correcta?

a) El detector de bordes de Canny se comporta como un filtro lineal dado que emplea filtrado Gaussiano para suavizar/emborronar la imagen y, a continuación, emplea filtrado lineal para calcular el gradiente.
b) El umbral óptimo en el algoritmo de Otsu se obtiene minimizando la varianza inter-clase o maximizando la varianza intra-clase.
c) En el algoritmo de Canny obtendremos bordes más discontinuos/fragmentados si reducimos el umbral bajo.
d) Las tres respuestas anteriores son erróneas.

Homework (exam quizzes, in Spanish)

¿Cuál de las afirmaciones siguientes sobre procesado de imagen es cierta?

a) Las derivadas en X detectan bordes horizontales, y las derivadas en Y detectan bordes verticales.

b) La magnitud del gradiente puede tener valores negativos, dado que el gradiente puede hacer referencia a pendientes positivas o negativas.

c) El gradiente en una imagen es mayor en las regiones planas que en las zonas con bordes.

d) Ninguna de las anteriores es correcta.

¿Cuáles de las siguientes afirmaciones relativas a la operación de convolución son correctas?

a) La convolución es equivariante con la traslación.

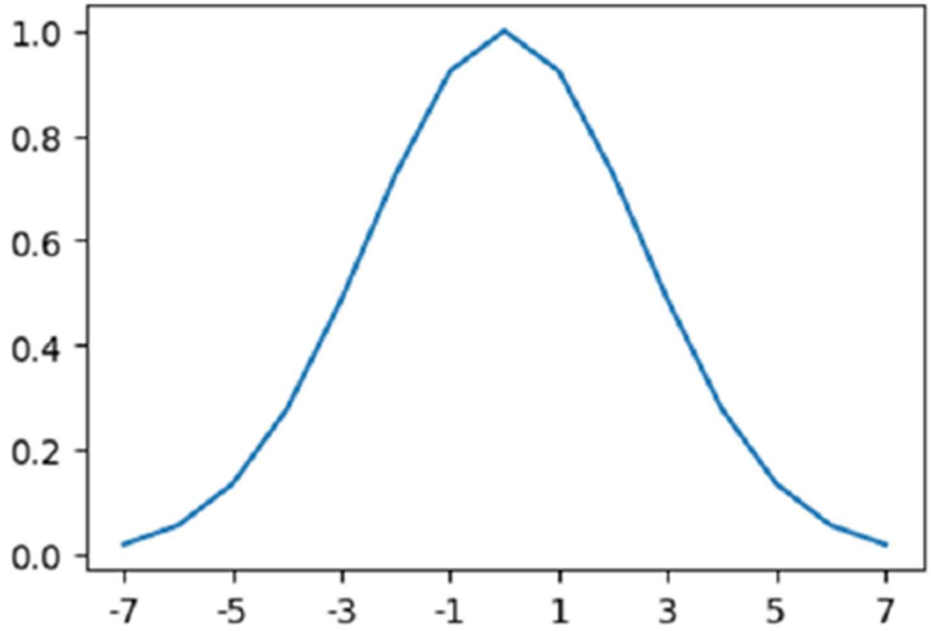
b) La convolución no es equivariante con el escalado.

c) La convolución no es equivariante con la rotación.

d) Todas las anteriores son correctas.

Homework (exam quizzes, in Spanish)

La siguiente figura representa el perfil de un kernel 1D. ¿Qué tipo de filtro representa?



a) Un kernel de suavizado Gaussiano.

b) Un kernel de primeras derivadas.

c) Un kernel de segundas derivadas.

d) Ninguno de los anteriores.

Gaussian Filtering and Edges

Pablo Mesejo

pmesejo@go.ugr.es

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD DE GRANADA





DaSCI

Instituto Andaluz de Investigación en **Data Science** and **Computational Intelligence**